Bazele inteligenței artificiale

Laborator – Grigore Ionuț

Laboratory contents

1. Introduction to Python
2. Linear models
3. Nonparametric models
4. Decision trees. Ensemble methods
5. Dimensionality reduction
6. Clustering
7. Test 1
8. Introduction to PyTorch
9. Multilayer perceptrons
10. Convolutional neural networks → **today's lab**
11. Recurrent neural networks
12. Attention mechanisms
13. Test 2
14. Recoveries

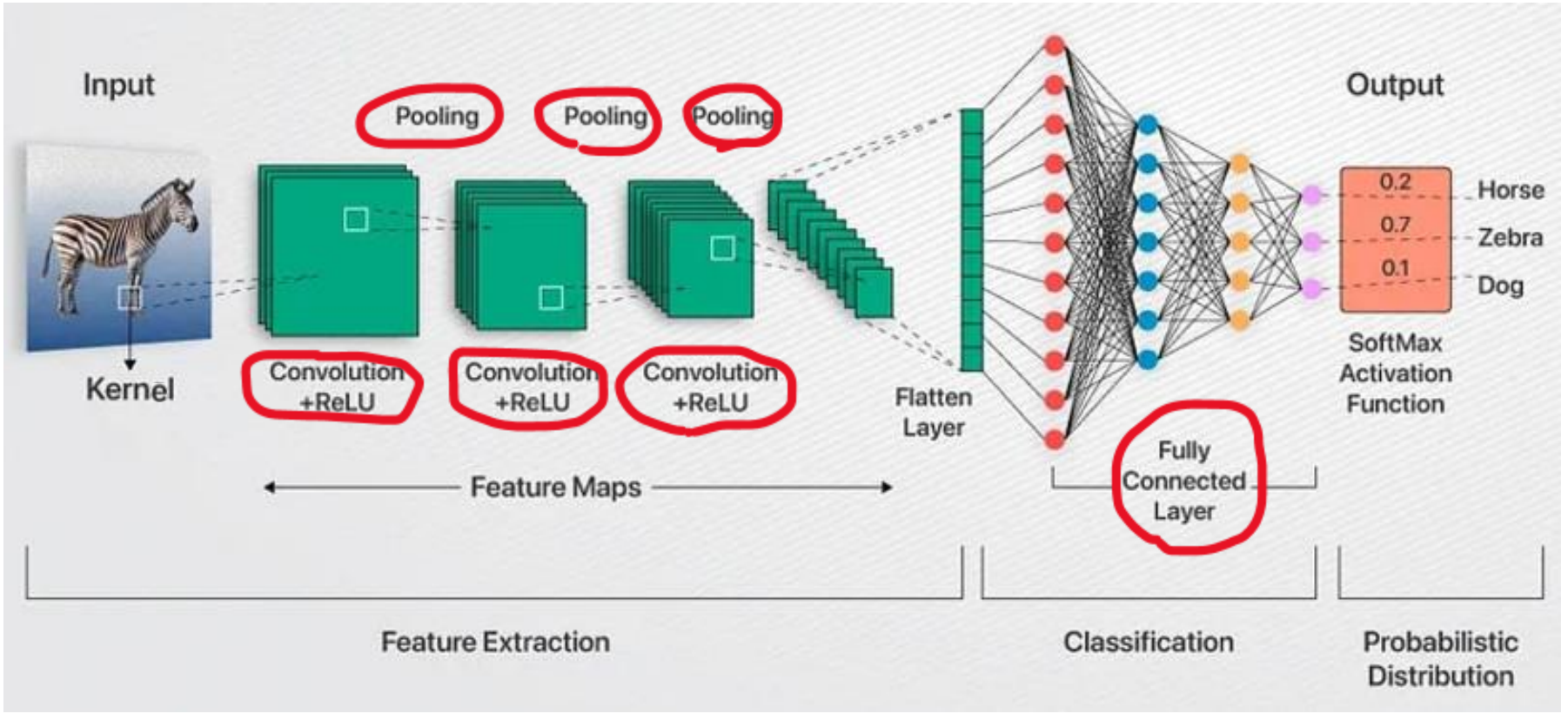
Content

- Convolutions for images
- Padding and stride
- Multiple input and multiple output channels
- Pooling
- LeNet
- AlexNet
- VGG
- Batch Normalization
- ResNet

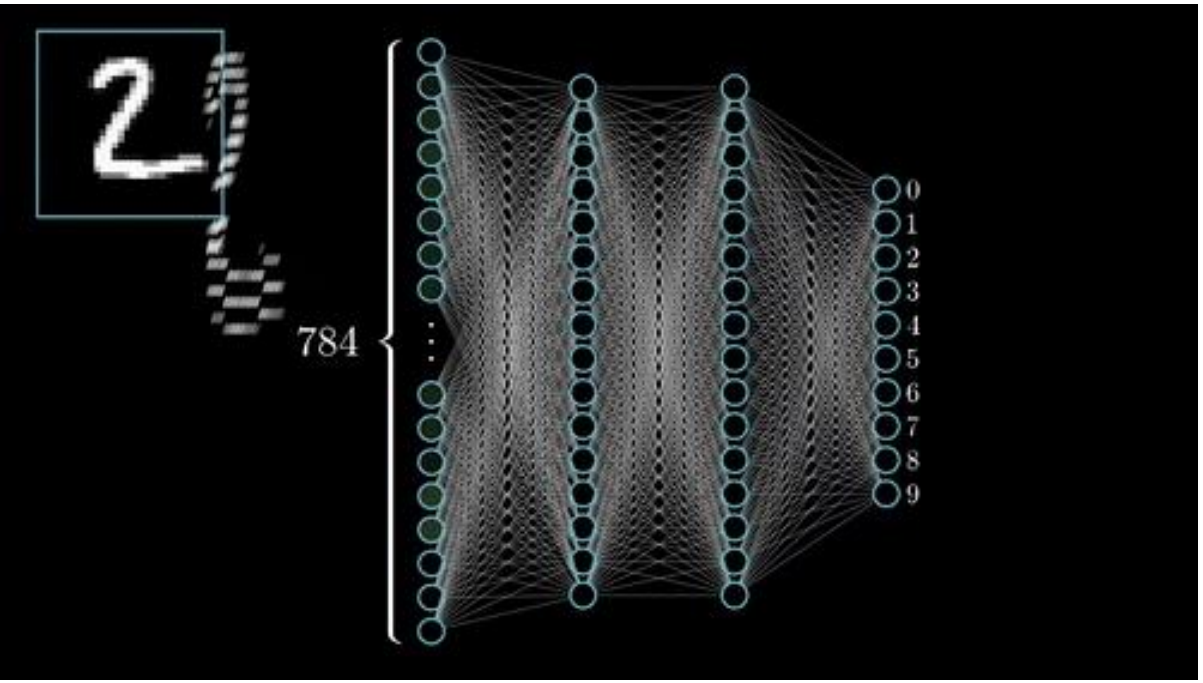
1) Convolutions for images

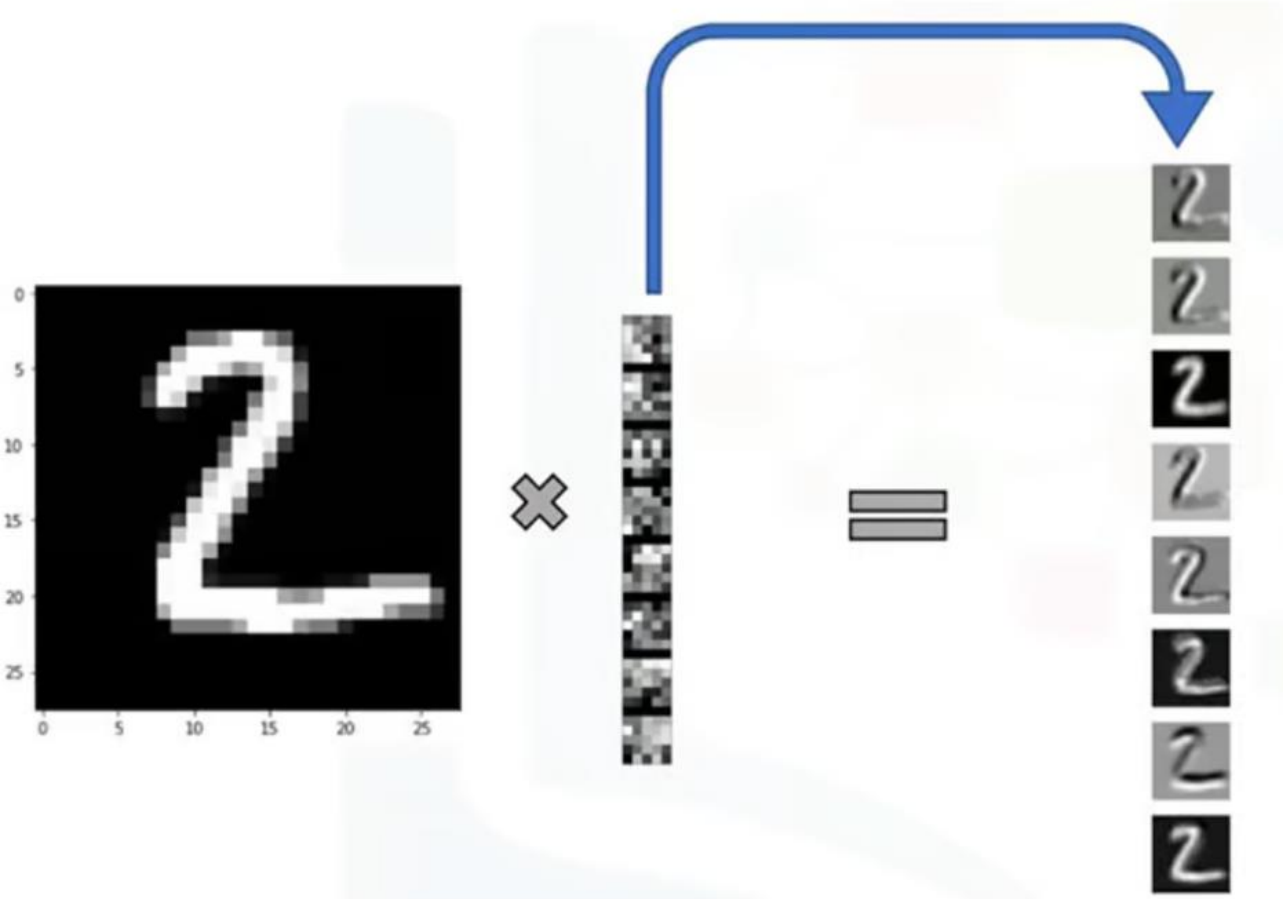
- a convolutional neural network (CNN) is a class of neural networks designed specifically for processing image data

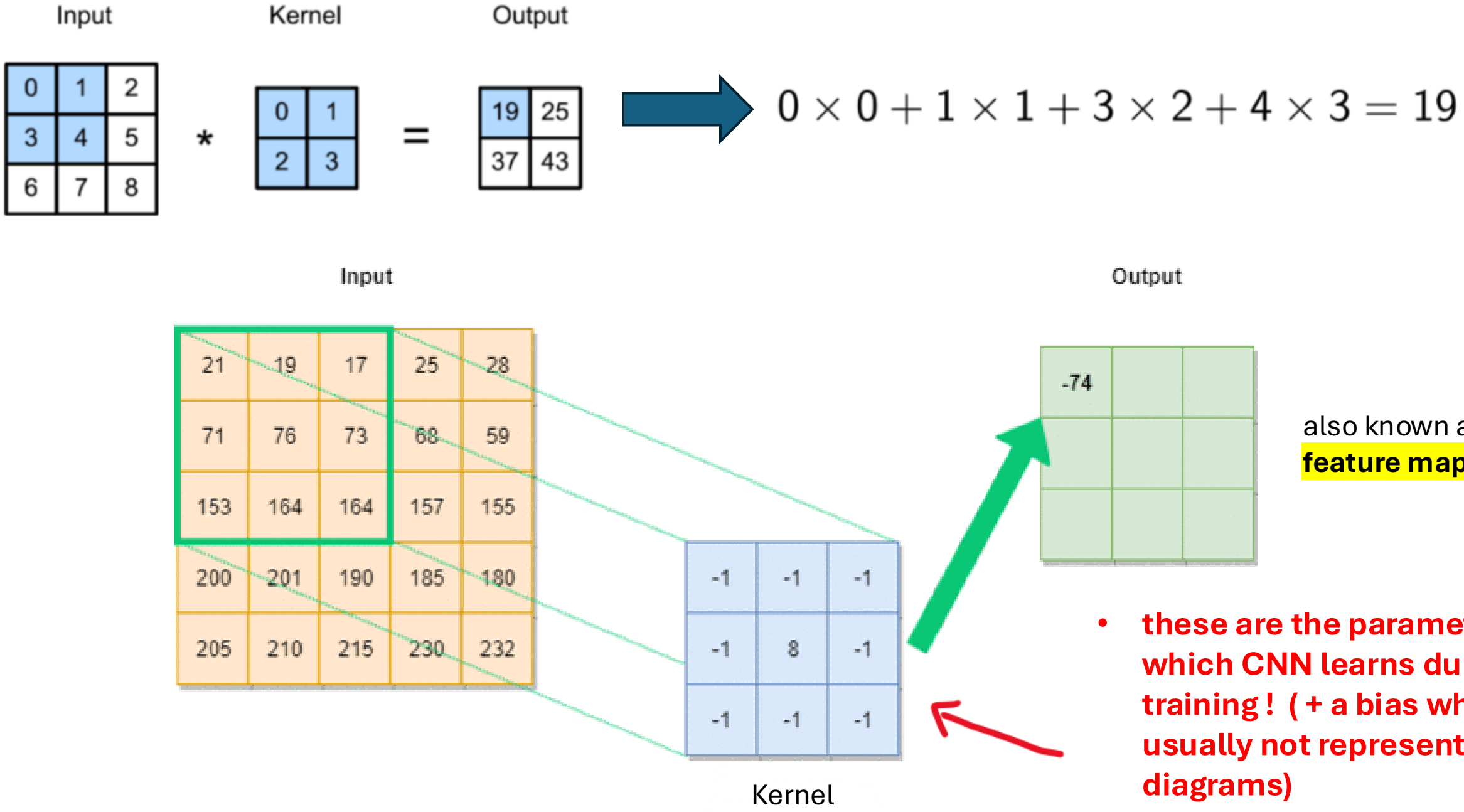
- A CNN typically includes 3 layers:
  - **convolutional layers**
  - **pooling layers**
  - **fully-connected layers** (from the previous lab)

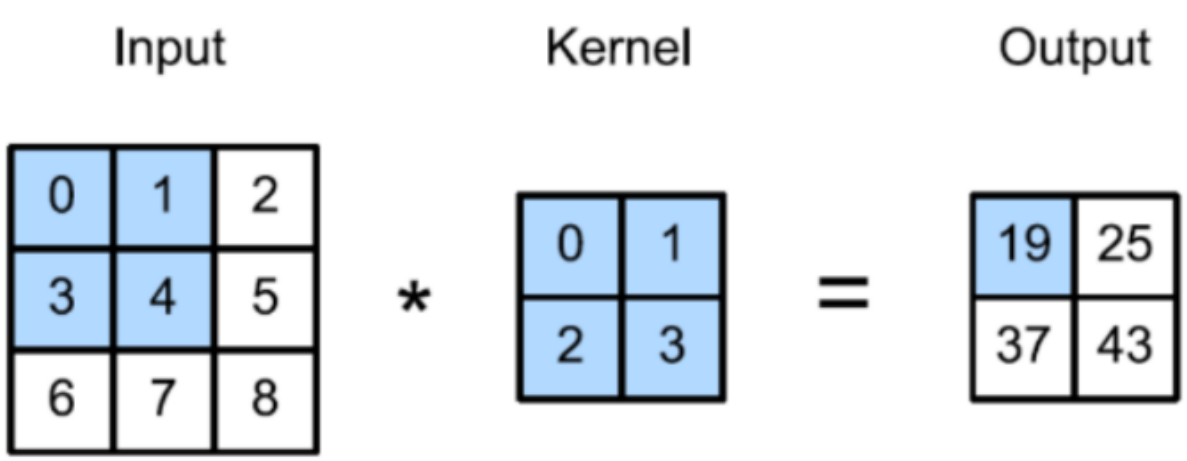


MLP vs CNN






Input
Kernel
Output
0 1 2
3 4 5
6 7 8
*
0 1
2 3
=
19 25
37 43
$0 \times 0 + 1 \times 1 + 3 \times 2 + 4 \times 3 = 19$
Input
Output
21 19 17 25 28
71 76 73 68 59
153 164 164 157 155
200 201 190 185 180
205 210 215 230 232
-1 -1 -1
-1 8 -1
-1 -1 -1
-74
Kernel
also known as
feature map
these are the parameters which CNN learns during training ! (+ a bias which is usually not represented in diagrams)
they are initially random

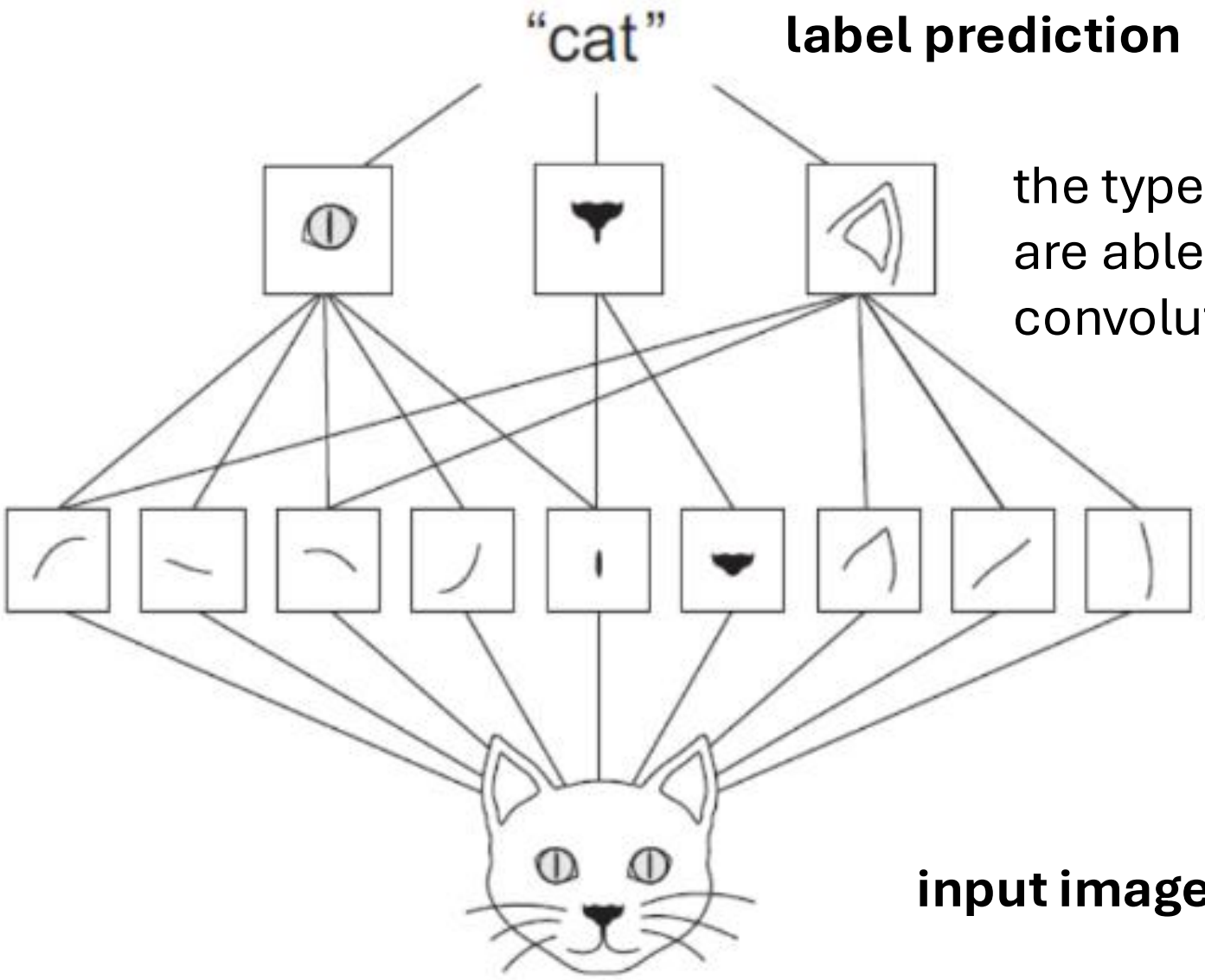


$$
\begin{cases}
19 = (0 \cdot 0) + (1 \cdot 1) + (3 \cdot 2) + (4 \cdot 3) \\
25 = (1 \cdot 0) + (2 \cdot 1) + (4 \cdot 2) + (5 \cdot 3) \\
37 = (3 \cdot 0) + (4 \cdot 1) + (6 \cdot 2) + (7 \cdot 3) \\
43 = (4 \cdot 0) + (5 \cdot 1) + (7 \cdot 2) + (8 \cdot 3)
\end{cases}
$$

```
X = torch.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]]]])
K = torch.tensor([[[[0.0, 1.0], [2.0, 3.0]]]])
F.conv2d(X, K)
```

```
tensor([[[[19., 25.],
          [37., 43.]]]])
```

Intuition behind CNNs: a simple example



label prediction

the types of features the kernels are able to detect in the second convolutional layer

the types of features the kernels are able to detect in the first convolutional layer

input image

MLP vs CNN (for images)

- **Architecture Design**
  - **MLP**:
    - an MLP is a **fully connected network** where every neuron in one layer is connected to every neuron in the next layer
    - input data is flattened into a **1D vector** before being passed to the MLP. For an image, this means losing the spatial structure (e.g., rows and columns)
  - **CNN**:
    - a CNN uses **convolutional layers** to detect patterns (like edges, textures) in local regions of the image, preserving spatial structure
    - it processes data in a grid-like form (e.g., a 2D image matrix) and extracts hierarchical features (see previous slide)
  - **Example**:
    - for a grayscale image of size 28×28 pixels:
      - **MLP**: The image is flattened into a 1×784 vector and fed to the network
      - **CNN**: The image remains 28×28, and the convolutional layers process it as-is, preserving spatial structure

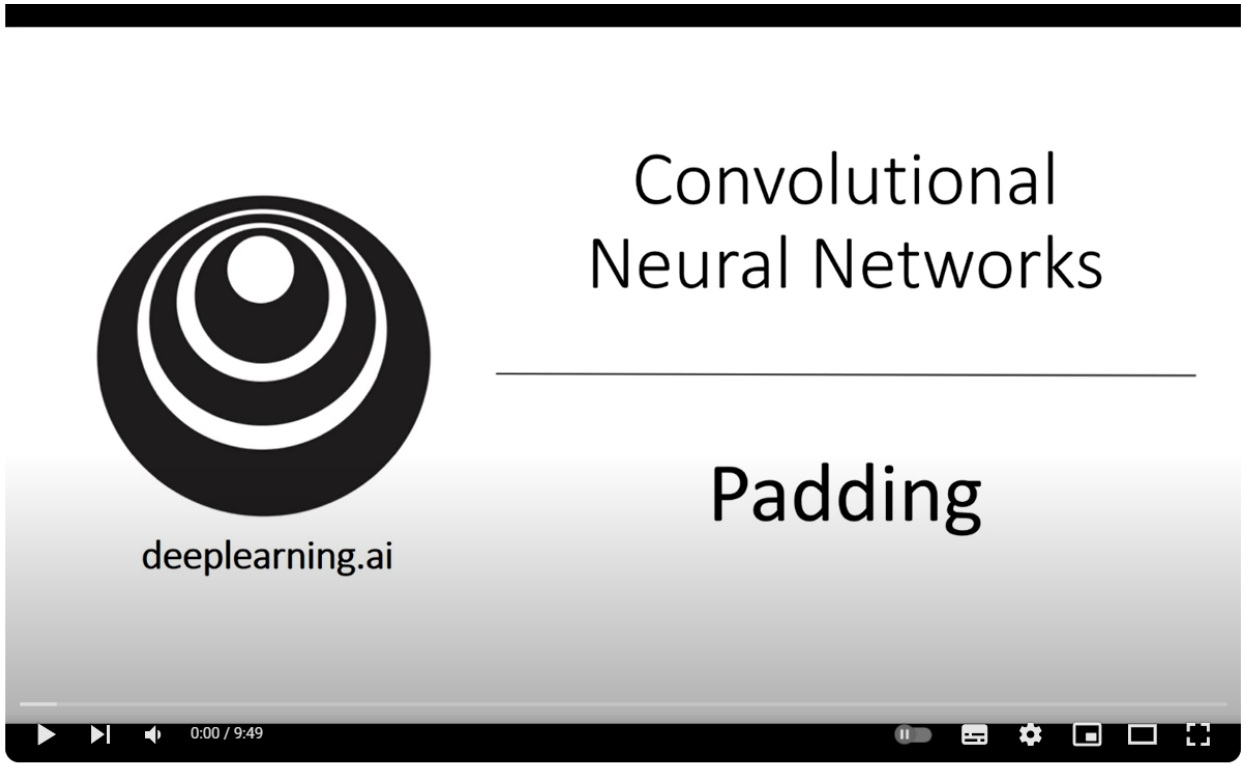
MLP vs CNN (for images)

- **Handling Spatial Information**
  - **MLP**:
    - since the image is flattened, it treats all pixels as independent features, ignoring spatial relationships (e.g., nearby pixels in a line forming an edge)
  - **CNN**:
    - a CNN uses convolutional filters to capture spatial patterns like edges, shapes, and textures. Pooling layers reduce spatial dimensions but retain important spatial features
  - **Example**:
    - A picture of a cat:
      - **MLP**: cannot distinguish between shuffled pixels and the original image because it doesn't consider spatial relationships
      - **CNN**: identifies spatial features like eyes, ears, and fur (=blană) texture due to its hierarchical structure

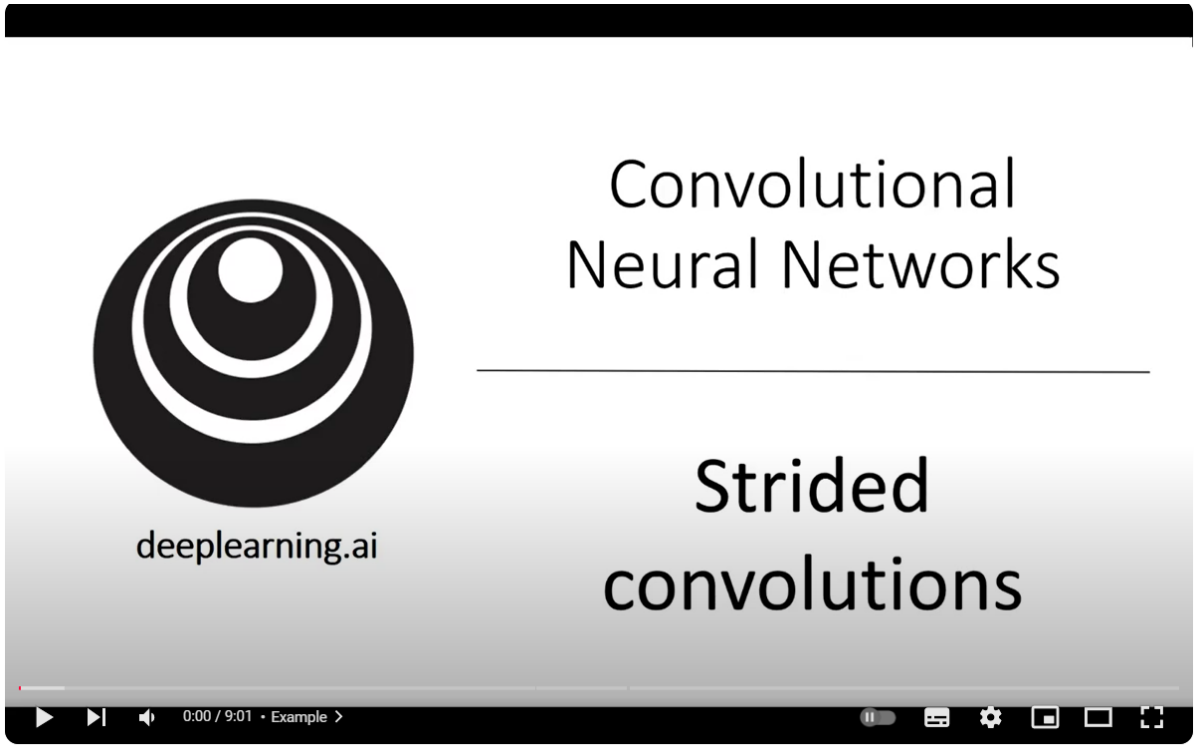
MLP vs CNN (for images)

- **Parameter Efficiency**
  - **MLP**:
    - requires many parameters because every input is connected to every neuron in the next layer, leading to a high computational cost, especially for large images
  - **CNN**:
    - uses shared weights (filters/kernels) across regions of the image, significantly reducing the number of parameters and making it more computationally efficient
  - **Example**:
    - for a grayscale image of size 100×100 pixels:
      - **MLP** with one hidden layer of 128 neurons: 100×100×128=1,280,000 parameters
      - **CNN** with a 3×3 filter: only 9 parameters per filter, applied across the image, if we have 128 filters, then will be 9×128=1152 parameters

2) Padding and stride



C4W1L04 Padding

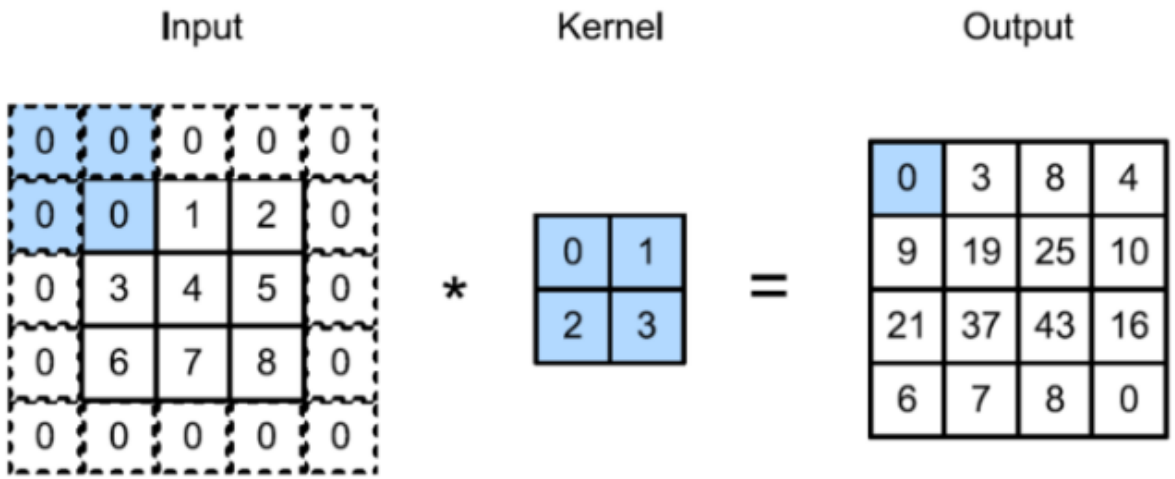


C4W1L05 Strided Convolutions

- the output shape of the convolutional layer is determined by the shape of the input and the shape of the kernel

- **padding**: addition of extra pixels around the boundary of the input image. Typically we set the values of the extra pixels to zero:

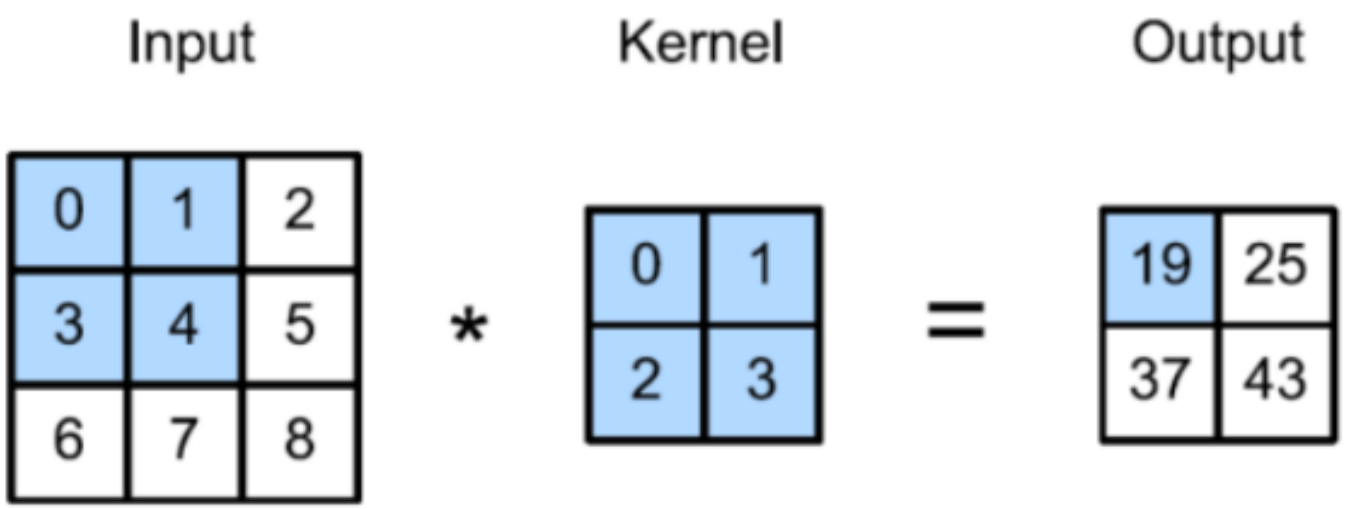
padding = 1 means that we apply 1 pixel of padding on all sides



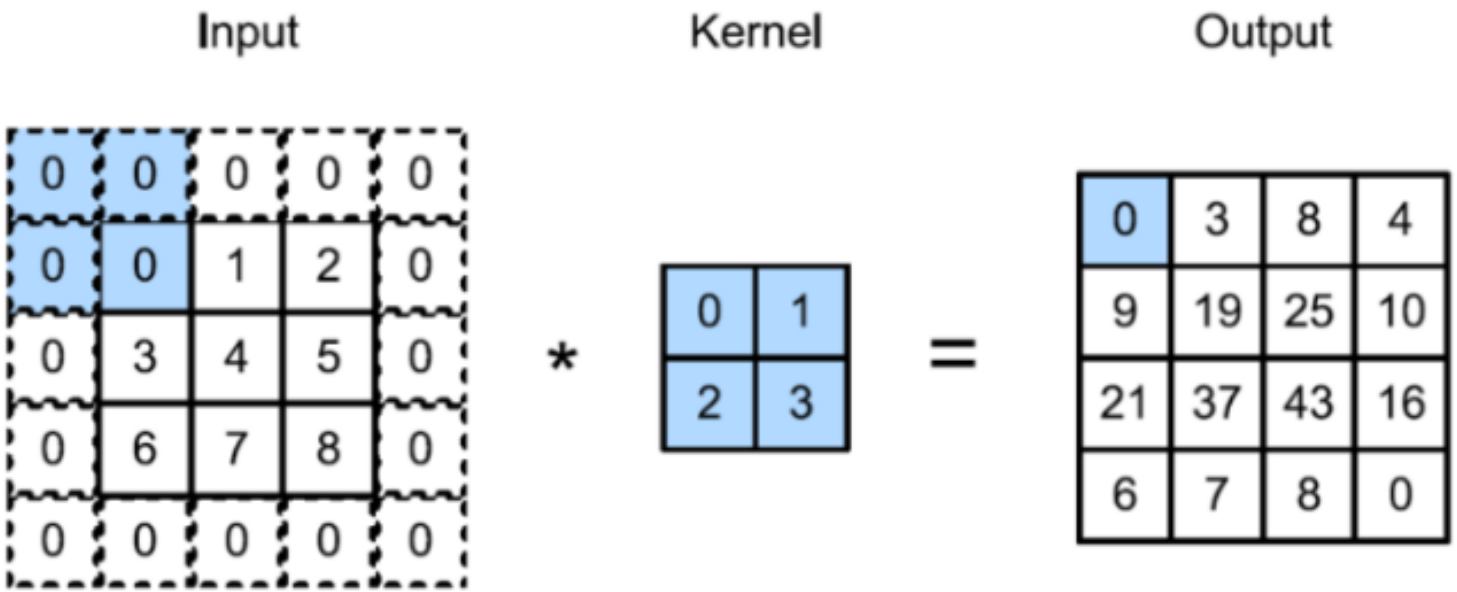

```
X = torch.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]]]])
K = torch.tensor([[[[0.0, 1.0], [2.0, 3.0]]]])
F.conv2d(X, K, padding=1)
```

```
tensor([[[[ 0.,  3.,  8.,  4.],
          [ 9., 19., 25., 10.],
          [21., 37., 43., 16.],
          [ 6.,  7.,  8.,  0.]]]])
```

without padding:



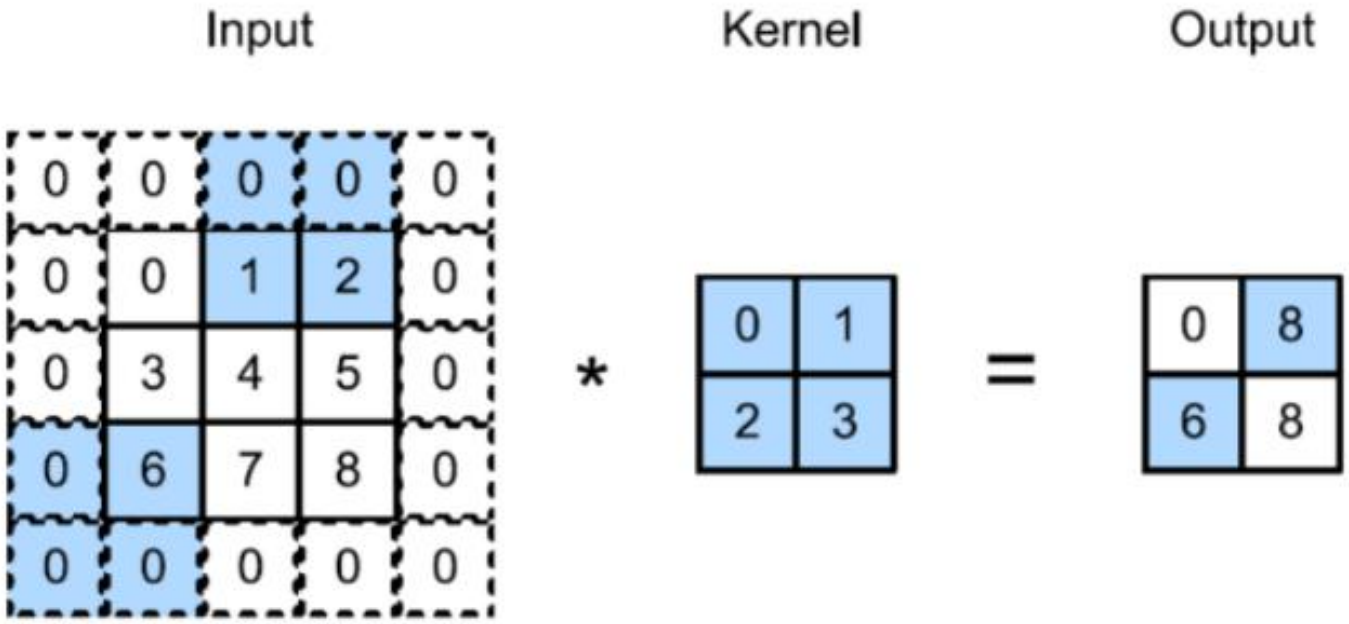
with padding:



- **stride**: the distance that the kernel moves over the input matrix:

it means that the kernel moves:

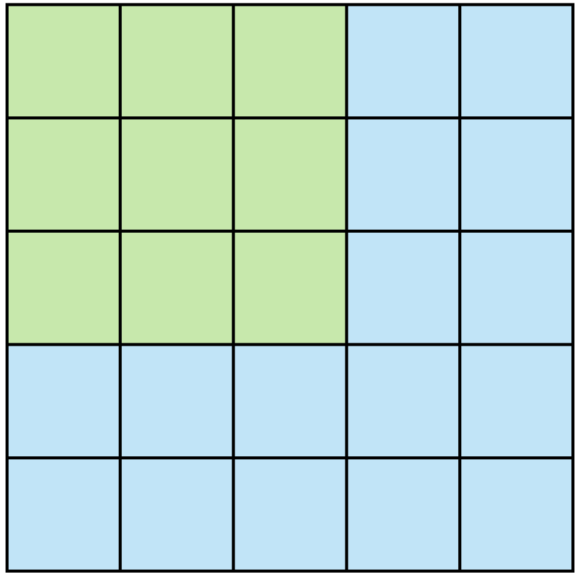
- 3 pixels vertically for each step
- 2 pixels horizontally for each step



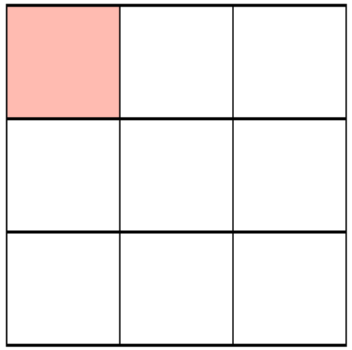

```
X = torch.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]]]])
K = torch.tensor([[[[0.0, 1.0], [2.0, 3.0]]]])
F.conv2d(X, K, padding=1, stride=(3, 2))
```

```
tensor([[[[0., 8.],
          [6., 8.]]]])
```

Example of stride 1 vs stride 2



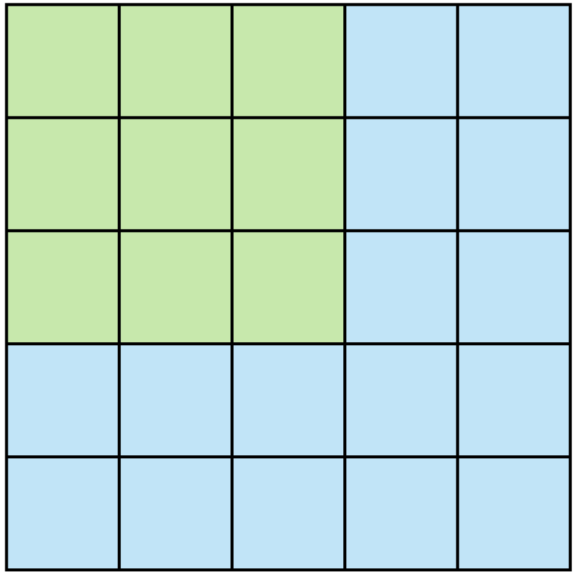
Stride 1



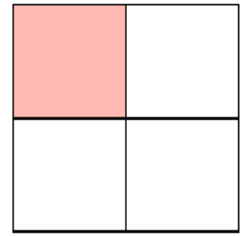
Feature Map

it means that the kernel moves:

- 1 pixel vertically for each step
- 1 pixel horizontally for each step



Stride 2

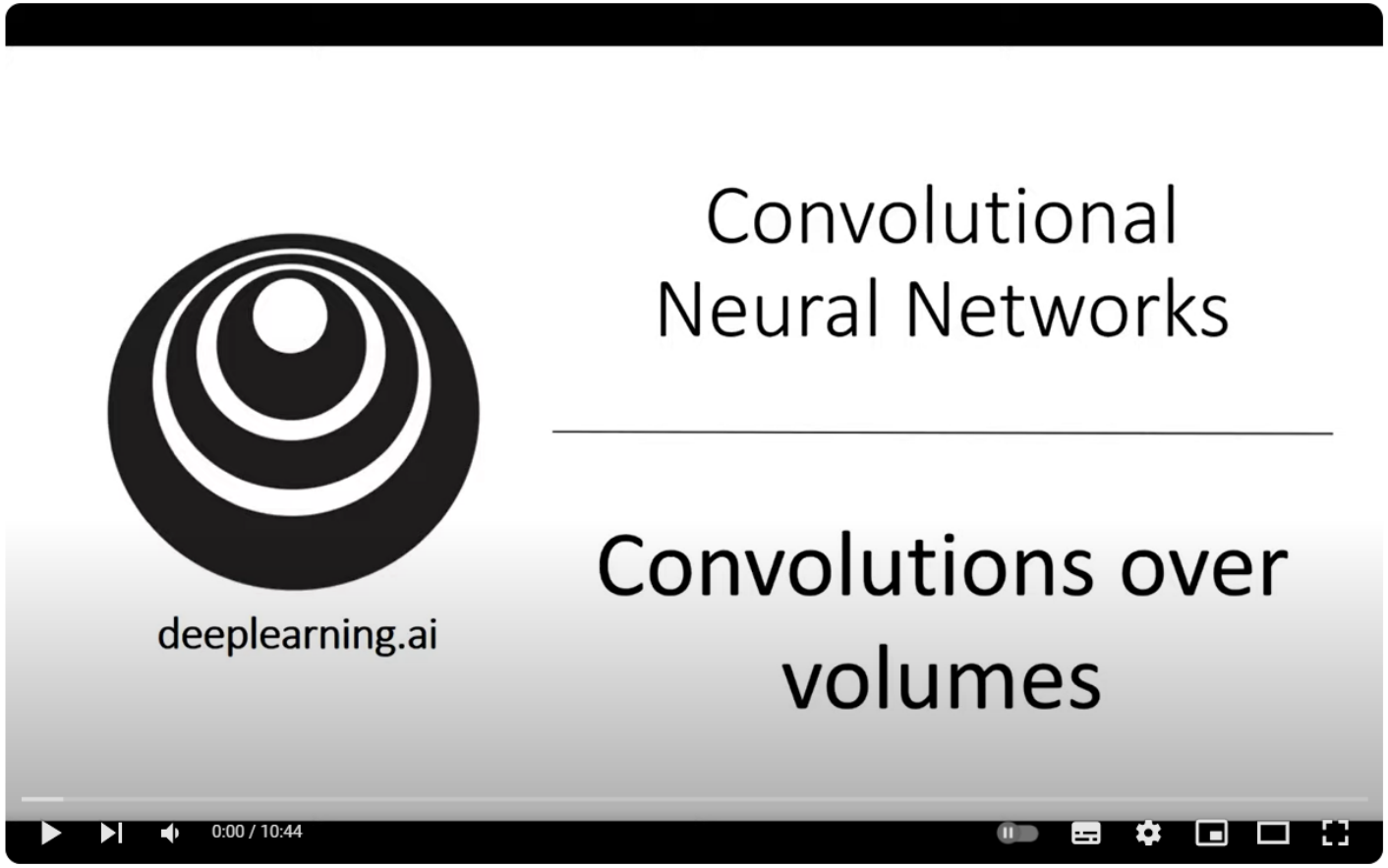


Feature Map

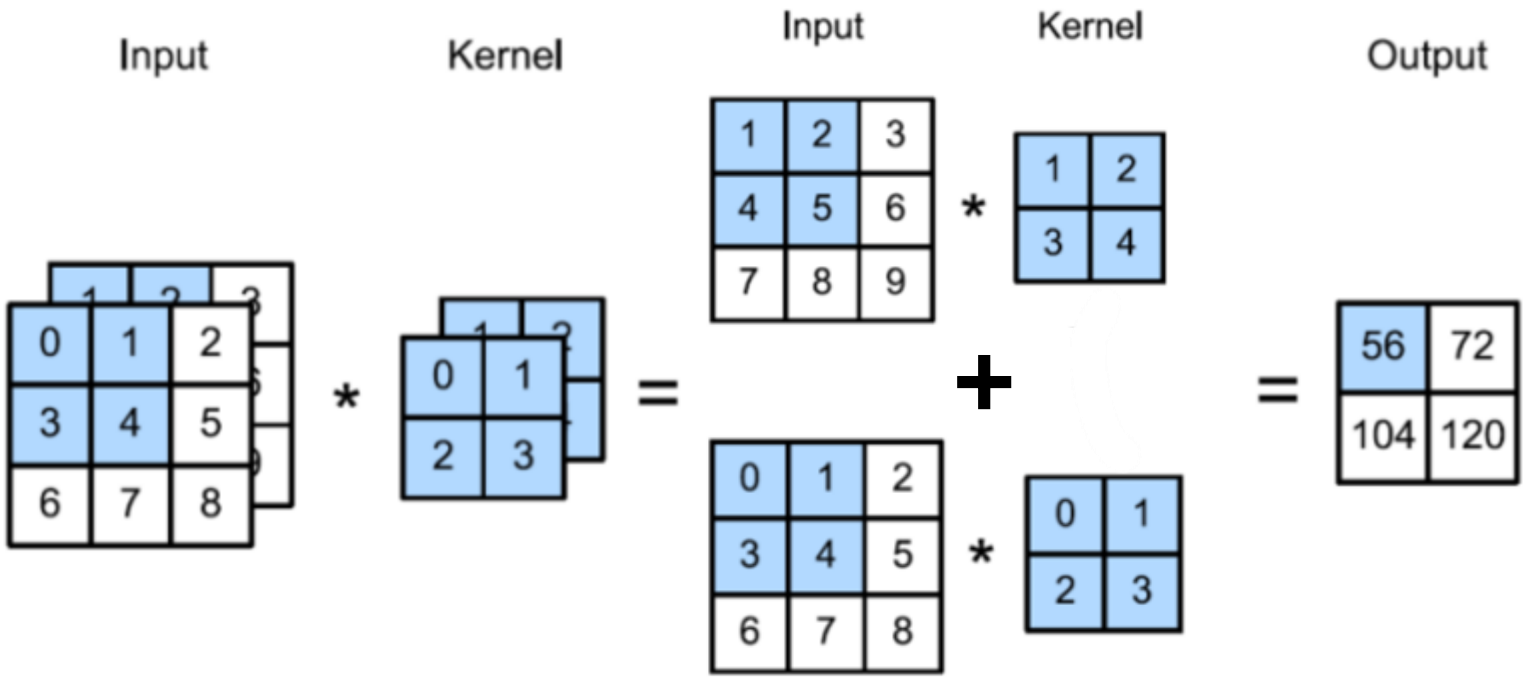
it means that the kernel moves:

- 2 pixels vertically for each step
- 2 pixels horizontally for each step

3) Multiple input and multiple output channels



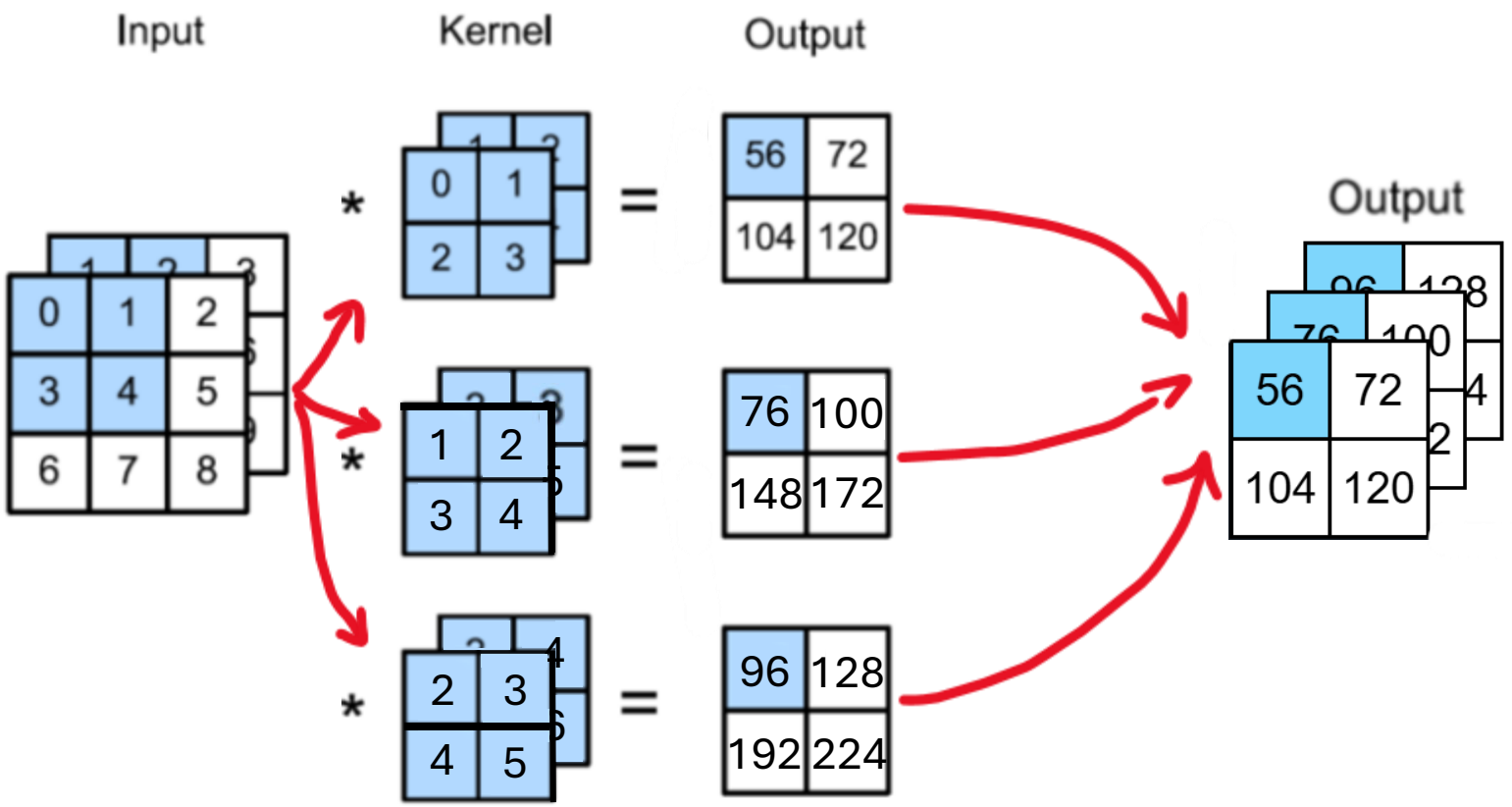
C4W1L06 Convolutions Over Volumes



$$
\begin{aligned}
56 &= [(1\cdot 1)+(2\cdot 2)+(4\cdot 3)+(5\cdot 4)]+[(0\cdot 0)+(1\cdot 1)+(3\cdot 2)+(4\cdot 3)]\\
72 &= [(2\cdot 1)+(3\cdot 2)+(5\cdot 3)+(6\cdot 4)]+[(1\cdot 0)+(2\cdot 1)+(4\cdot 2)+(5\cdot 3)]\\
104 &= [(4\cdot 1)+(5\cdot 2)+(7\cdot 3)+(8\cdot 4)]+[(3\cdot 0)+(4\cdot 1)+(6\cdot 2)+(7\cdot 3)]\\
120 &= [(5\cdot 1)+(6\cdot 2)+(8\cdot 3)+(9\cdot 4)]+[(4\cdot 0)+(5\cdot 1)+(7\cdot 2)+(8\cdot 3)]
\end{aligned}
$$

```
X = torch.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]],
                  [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]]]])
K = torch.tensor([[[[0.0, 1.0], [2.0, 3.0]], [[1.0, 2.0], [3.0, 4.0]]]])
F.conv2d(X, K)
```

```
tensor([[[[ 56.,  72.],
          [104., 120.]]]])
```



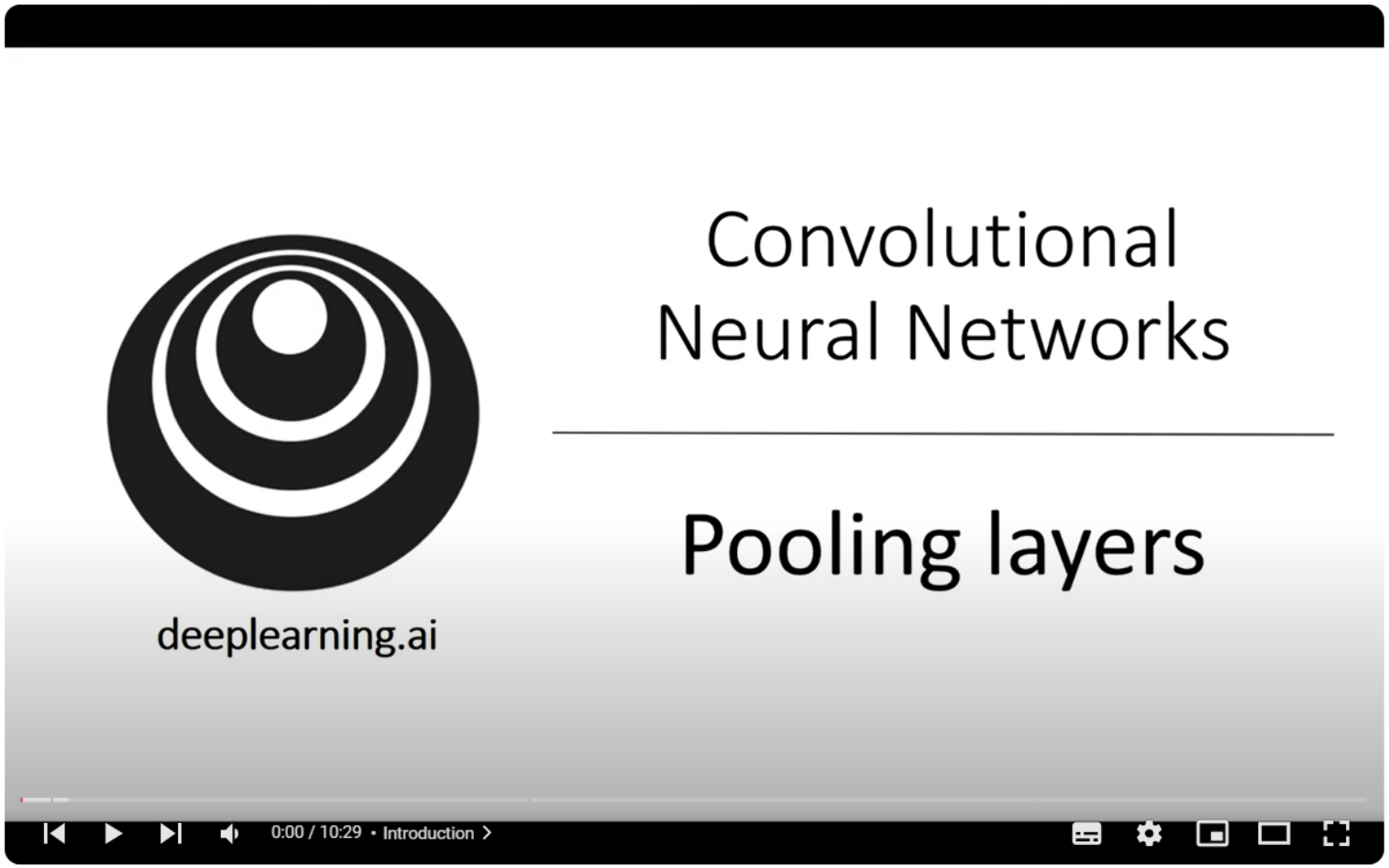

```
X = torch.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]],
                  [[1.0, 2.0, 3.0], [4.0, 5.0, 6.0], [7.0, 8.0, 9.0]]]])
K = torch.tensor([[[[0.0, 1.0], [2.0, 3.0]], [[1.0, 2.0], [3.0, 4.0]]]])
K = torch.cat((K, K + 1, K + 2))
F.conv2d(X, K)
```

```
tensor([[[[ 56.,  72.],
          [104., 120.]],

         [[ 76., 100.],
          [148., 172.]],

         [[ 96., 128.],
          [192., 224.]]]])
```

4) Pooling



C4W1L09 Pooling Layers

- pooling reduces the spatial size of the feature maps

- this decreases the number of parameters and computations in subsequent layers, making the network more computationally efficient

- **it contains no parameters (there are no learnable parameters for it)**

- we can have:
  - **Max Pooling**: retains the maximum value in each region
  - **Average Pooling**: computes the average value of all activations in each region
  - etc.

Input

0	1	2
3	4	5
6	7	8

2 x 2 Max Pooling

Output

4	5
7	8

$$\max(0,1,3,4) = 4$$
$$\max(1,2,4,5) = 5$$
$$\max(3,4,6,7) = 7$$
$$\max(4,5,7,8) = 8$$

```
X = torch.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]]]])
F.max_pool2d(X, kernel_size=2, stride=1)
```

```
tensor([[[[4., 5.],
          [7., 8.]]]])
```

Input

0	1	2
3	4	5
6	7	8

2x2 Average Pooling

Output

2	3
5	6

$$\frac{0+1+3+4}{4} = \frac{8}{4} = 2$$
$$\frac{1+2+4+5}{4} = \frac{12}{4} = 3$$
$$\frac{3+4+6+7}{4} = \frac{20}{4} = 5$$
$$\frac{4+5+7+8}{4} = \frac{24}{4} = 6$$

```
X = torch.tensor([[[[0.0, 1.0, 2.0], [3.0, 4.0, 5.0], [6.0, 7.0, 8.0]]]])
F.avg_pool2d(X, kernel_size=2, stride=1)
```

```
tensor([[[[2., 3.],
          [5., 6.]]]])
```

Feature Map

6	6	6	6
4	5	5	4
2	4	4	2
2	4	4	2

Max Pooling

Average Pooling

Benefits of using pooling layers

Dimensionality Reduction

- pooling reduces the spatial dimensions (height and width) of feature maps
- this decreases the number of parameters and computations in subsequent layers, making the network more efficient and reducing the risk of overfitting

Feature Selection

- pooling layers distill the most significant features in a region, such as:
  - **Max Pooling**: retains the strongest activations (most important features)
  - **Average Pooling**: averages activations to provide a smoother representation
- this ensures that only the most salient features are propagated further in the network

5) LeNet

- LeNet or LeNet-5 is a pioneering convolutional neural network (CNN) architecture designed by **Yann LeCun** and his collaborators in 1998



→ "Godfather of AI", actually Chief AI Scientist at Meta

- it was originally developed for handwritten digit recognition, such as digits in ZIP codes

- **NOTE**: you don't need to memorize the architecture, it's just to know that **LeNet is widely recognized as the first working CNN in history**

- at the time of its development, deep learning and CNNs were in their beginning. There was no established set of best practices or guidelines. Yann LeCun and his team designed LeNet by trial and error, experimenting with various configurations to optimize performance for their specific task: handwritten digit recognition

What makes LeNet so special?

- it was the first working CNN: introduced the world to the concept of CNNs for image recognition tasks

- many core ideas in modern CNNs were first introduced or implemented in LeNet:
  - **convolutional layers**
  - **pooling layers**
  - **hierarchical feature learning**: LeNet used a hierarchical approach, where lower layers learned simple features (like edges), and deeper layers combined them into more complex patterns (like shapes or digits). This hierarchical learning is a cornerstone of modern deep learning

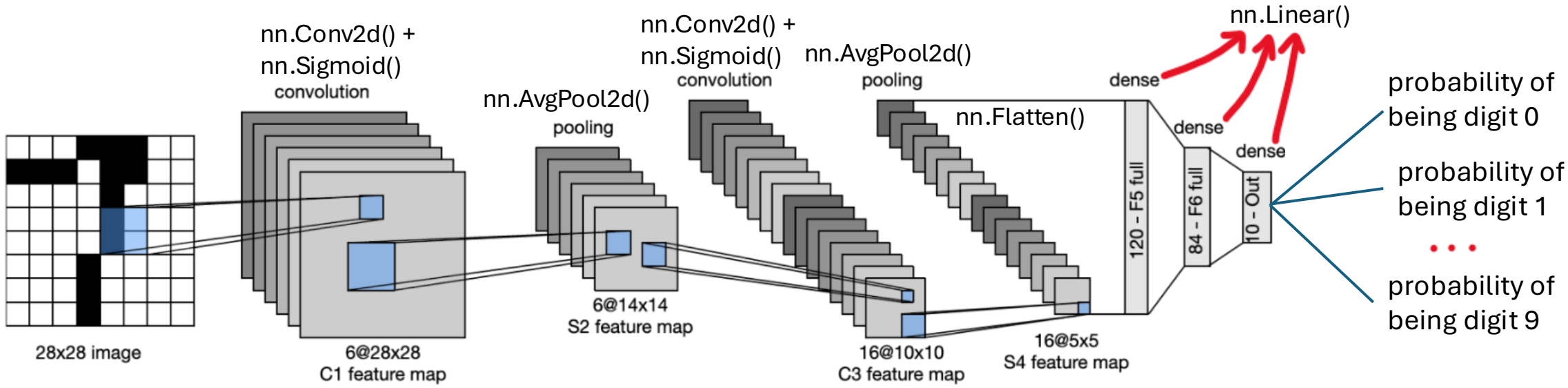
- it was one of the first neural networks to demonstrate that **backpropagation** and **gradient-based optimization** could train neural networks effectively

```python
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 6 * 6, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))
```

you don't need to memorize it ☺

```python
X = torch.rand(1, 1, 28, 28)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
Conv2d output shape:     torch.Size([1, 6, 28, 28])
Sigmoid output shape:    torch.Size([1, 6, 28, 28])
AvgPool2d output shape:  torch.Size([1, 6, 14, 14])
Conv2d output shape:     torch.Size([1, 16, 10, 10])
Sigmoid output shape:    torch.Size([1, 16, 10, 10])
AvgPool2d output shape:  torch.Size([1, 16, 5, 5])
Flatten output shape:    torch.Size([1, 400])
Linear output shape:     torch.Size([1, 120])
Sigmoid output shape:    torch.Size([1, 120])
Linear output shape:     torch.Size([1, 84])
Sigmoid output shape:    torch.Size([1, 84])
Linear output shape:     torch.Size([1, 10])
```



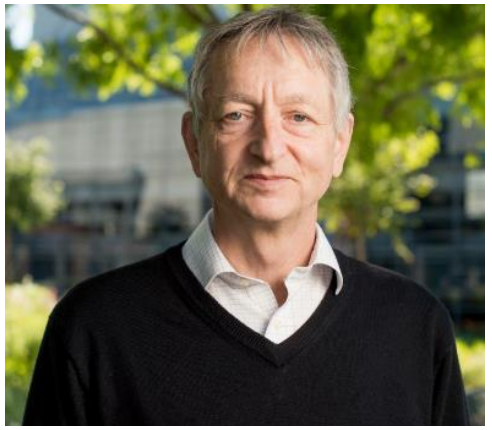
AT&T
LeNet 5
RESEARCH
answer: 0
0
103

6) AlexNet

- developed by **Alex Krizhevsky** and **Geoffrey Hinton** in 2012

he is Alex ☺ ←





→

- "Godfather of AI" together with Yann LeCun
- he won the Nobel Prize in 2024

- AlexNet is a deep convolutional neural network designed to classify images into 1000 categories in the **ImageNet** dataset



→

about ImageNet:

- number of images: over 14 million images
- number of categories: more than 20,000 categories
- image sizes: varying resolutions, with most images being of sufficient quality for recognition tasks

- its architecture builds on concepts from LeNet but scales them up significantly to handle the complexity of the ImageNet dataset

What makes AlexNet so special?

- **winning the ImageNet Challenge** (2012): AlexNet achieved a top-5 error rate of **15.3%**, significantly outperforming the second-place model (26.2%). This dramatic improvement highlighted the potential of deep learning for solving complex visual recognition tasks

- **it was the first architecture to successfully scale deep learning to a large dataset like ImageNet**, which contains over 1.2 million high-resolution images across 1000 classes. This demonstrated that deep learning could be applied to large-scale problems effectively, given sufficient computational resources

- **it was the first major deep learning architecture to extensively use GPUs for training**, making it feasible to train a large, deep network on the massive ImageNet dataset

- AlexNet introduced or popularized several key innovations in deep learning:
  - **ReLU Activation Function (NOTE:** LeNet was using sigmoid**)**
  - **Dropout Regularization**
  - **Data Augmentation**

ImageNet Large Scale Visual Recognition Challenge (ILSVRC)

1. **What is ILSVRC?**
    - A subset of ImageNet with 1,000 categories and 1.2 million training images is used for the annual **ImageNet Large Scale Visual Recognition Challenge** (ILSVRC).
    - Tasks include:
        - Image classification: Classifying images into one of 1,000 categories.
        - Object detection: Detecting and localizing objects in images.
        - Object localization: Identifying both the class and bounding box of an object.
2. **Significance of ILSVRC**:
    - ILSVRC has driven advances in computer vision and deep learning, serving as a benchmark for new architectures and algorithms.
    - Winning models like **AlexNet (2012)**, **VGG (2014)**, **ResNet (2015)**, and **EfficientNet (2019)** have become milestones in deep learning history.

The **top-5 error rate** is a metric used in multi-class classification problems, especially in computer vision tasks like the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)**.

How It Works

For each test image:

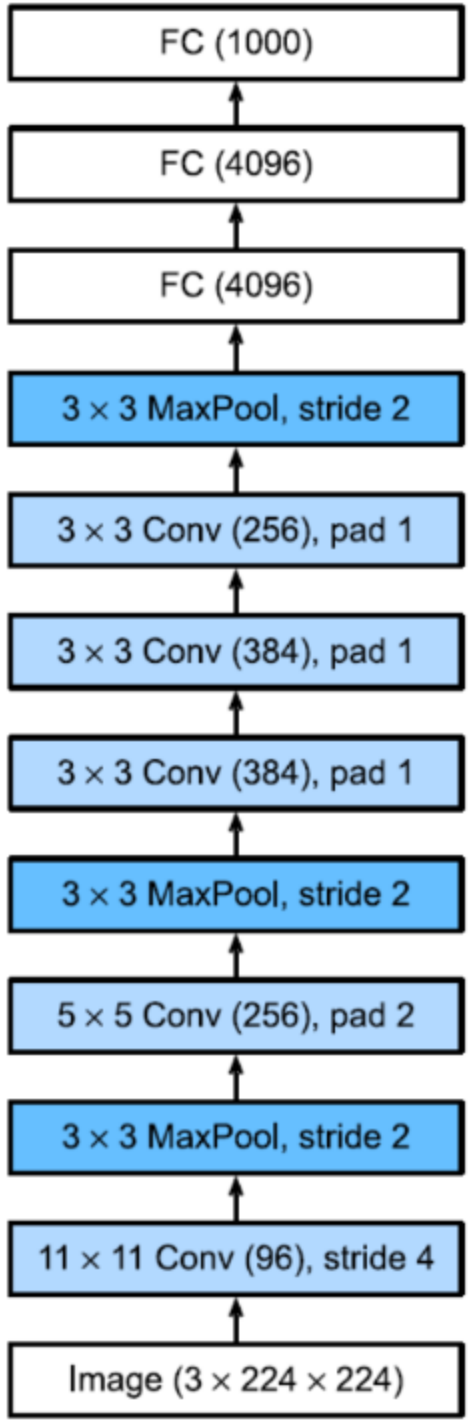
1. The model outputs a ranked list of class probabilities (e.g., softmax scores) for all possible classes.
2. The **top-5 predictions** are the 5 classes with the highest probabilities.
3. If the true class is **within these top-5 predictions**, it is counted as correct. Otherwise, it is considered an error.

The **top-5 error rate** is computed as:

$$\text{Top-5 Error Rate} = \frac{\text{Number of images where true label is not in top-5 predictions}}{\text{Total number of images in the dataset}}$$

```python
net = nn.Sequential(
    # Here, we use a larger 11 x 11 window to capture objects. At the same
    # time, we use a stride of 4 to greatly reduce the height and width of the
    # output. Here, the number of output channels is much larger than that in
    # LeNet
    nn.Conv2d(1, 96, kernel_size=11, stride=4, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    # Make the convolution window smaller, set padding to 2 for consistent
    # height and width across the input and output, and increase the number of
    # output channels
    nn.Conv2d(96, 256, kernel_size=5, padding=2), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    # Use three successive convolutional layers and a smaller convolution
    # window. Except for the final convolutional layer, the number of output
    # channels is further increased. Pooling layers are not used to reduce the
    # height and width of input after the first two convolutional layers
    nn.Conv2d(256, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 384, kernel_size=3, padding=1), nn.ReLU(),
    nn.Conv2d(384, 256, kernel_size=3, padding=1), nn.ReLU(),
    nn.MaxPool2d(kernel_size=3, stride=2),
    nn.Flatten(),
    # Here, the number of outputs of the fully-connected layer is several
    # times larger than that in LeNet. Use the dropout layer to mitigate
    # overfitting
    nn.Linear(6400, 4096), nn.ReLU(),
    nn.Dropout(p=0.5),
    nn.Linear(4096, 4096), nn.ReLU(),
    nn.Dropout(p=0.5),
    # Output layer. Since we are using Fashion-MNIST, the number of classes is
    # 10, instead of 1000 as in the paper
    nn.Linear(4096, 10))
```

you don't need to memorize it ☺



7) VGG

What makes VGG so special?

- **uniform design**: VGGNet's architecture is elegant and simple, consisting of a series of stacked convolutional layers with **fixed filter sizes (3×3) and pooling layers (2×2):** showed that using **small kernels (3×3)** achieve the same or better performance than larger kernels

- **deep network**: VGGNet explored the effects of depth in neural networks by introducing architectures with **16** and **19 layers**, much deeper than predecessors like AlexNet (8 layers)

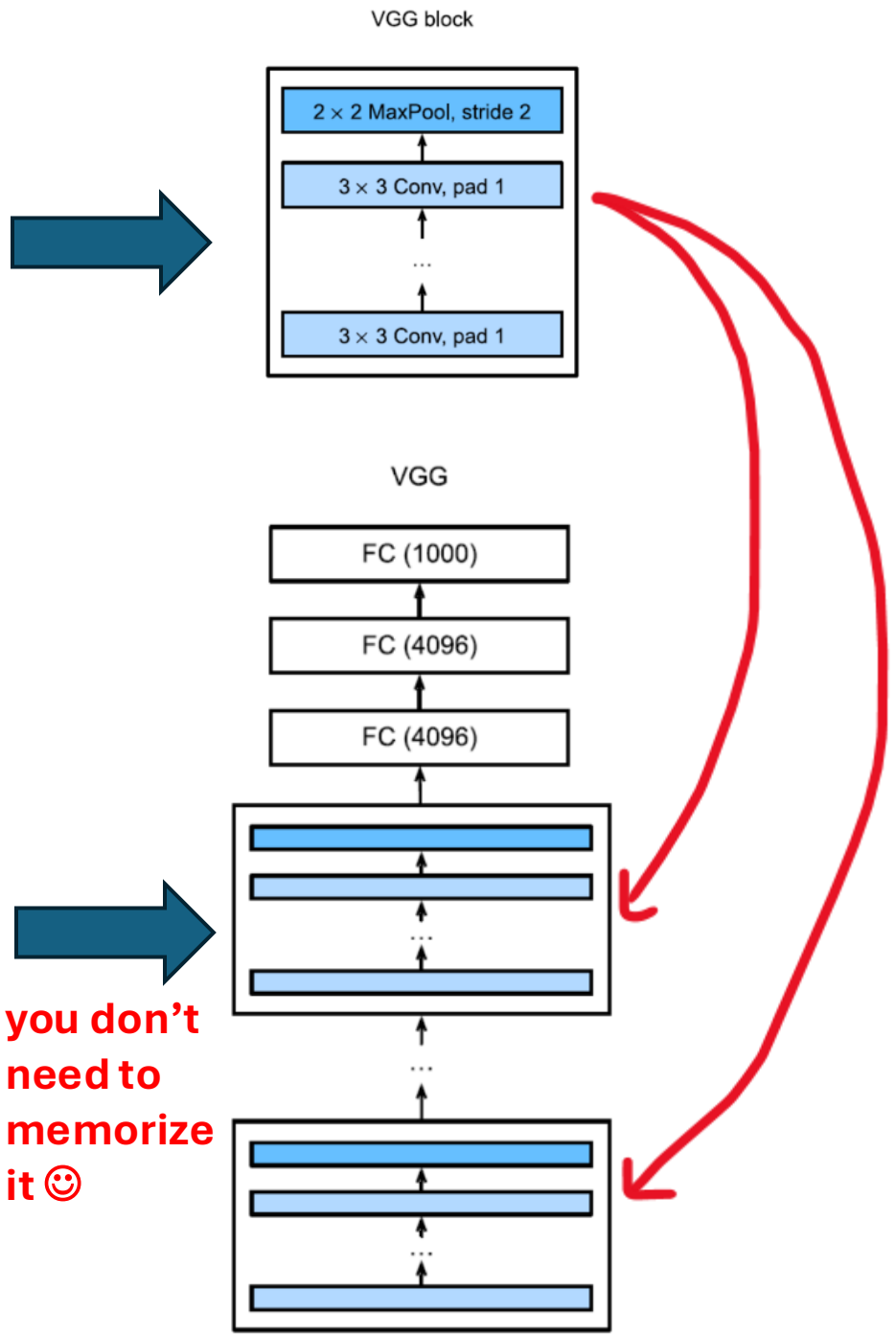
- **modularity**: the network is built from repeating blocks of convolution and pooling, making it straightforward to extend or modify for various tasks

```python
def vgg_block(num_convs, in_channels, out_channels):
    layers = []
    for _ in range(num_convs):
        layers.append(nn.Conv2d(in_channels, out_channels,
                                kernel_size=3, padding=1))
        layers.append(nn.ReLU())
        in_channels = out_channels
    layers.append(nn.MaxPool2d(kernel_size=2,stride=2))
    return nn.Sequential(*layers)
```

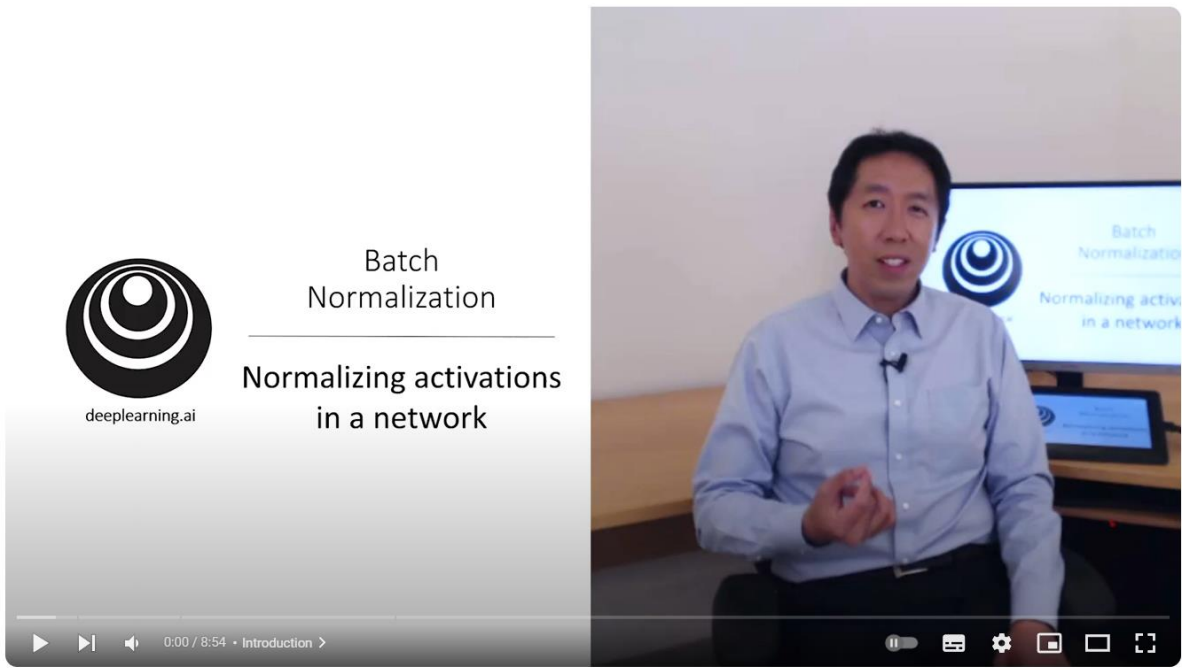
```python
def vgg(conv_arch):
    conv_blks = []
    in_channels = 1
    # The convolutional part
    for (num_convs, out_channels) in conv_arch:
        conv_blks.append(vgg_block(num_convs, in_channels, out_channels))
        in_channels = out_channels

    return nn.Sequential(
        *conv_blks, nn.Flatten(),
        # The fully-connected part
        nn.Linear(out_channels * 7 * 7, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 4096), nn.ReLU(), nn.Dropout(0.5),
        nn.Linear(4096, 10))

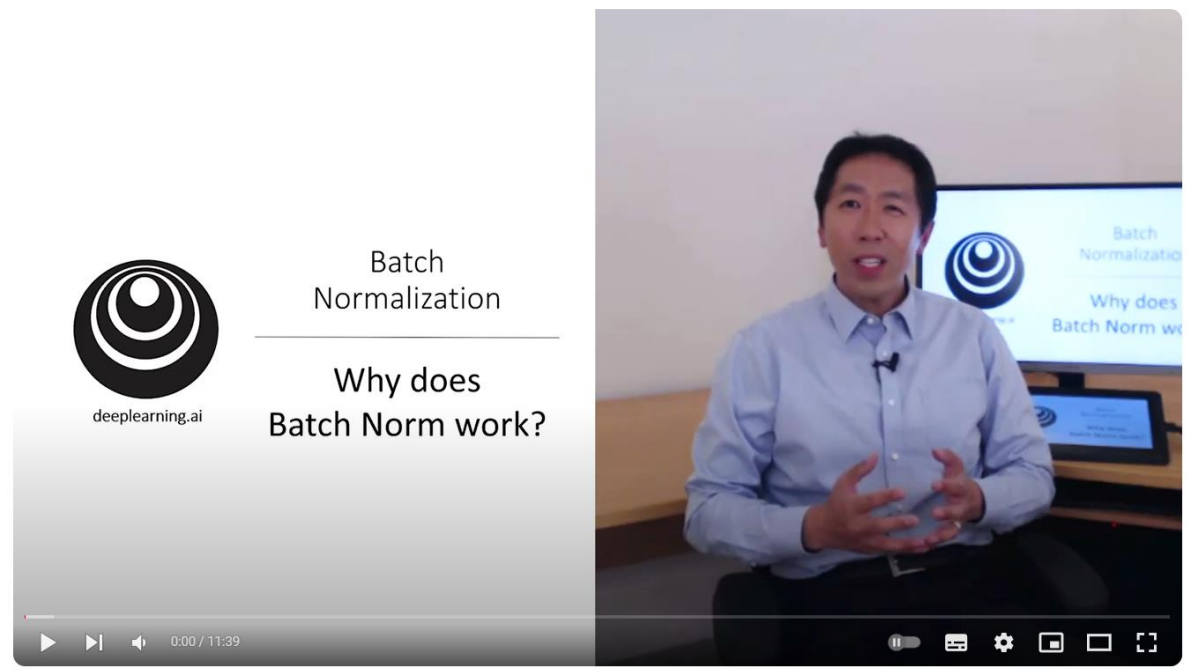
net = vgg(conv_arch)
```



8) Batch Normalization



Normalizing Activations in a Network (C2W3L04)



Why Does Batch Norm Work? (C2W3L06)

- is a technique introduced to stabilize and accelerate the training of deep neural networks

- is applied to individual layers (*nn.Linear*, *nn.Conv2d*) and works as follows:
  - in each training iteration, we first normalize the inputs by subtracting their mean and dividing by their standard deviation, where both are estimated based on the statistics of the current mini-batch
  - next, we apply a scale coefficient and a scale offset

$$BN(x) = \gamma \odot \frac{x - \hat{\mu}_B}{\hat{\sigma}_B} + \beta$$

Where:

1. x: The input to Batch Normalization (from a mini-batch $\mathcal{B}$).
2. $\hat{\mu}_B$: The mean of the inputs x in the mini-batch.
3. $\hat{\sigma}_B$: The standard deviation of the inputs x in the mini-batch.
4. γ: A learnable scale parameter.
5. β: A learnable shift parameter.
6. ϵ: A small constant to prevent division by zero when computing the standard deviation.

Step 1: Compute the Mini-Batch Statistics

1. **Mean ($\hat{\mu}_B$)**:
    - Calculate the mean of the inputs x for the mini-batch:

$$\hat{\mu}_B = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} x$$

    - This provides the average value of the feature x across the batch, centering the data around zero.
2. **Variance ($\hat{\sigma}_B^2$)**:
    - Calculate the variance of the inputs x for the mini-batch:

$$\hat{\sigma}_B^2 = \frac{1}{|\mathcal{B}|} \sum_{x \in \mathcal{B}} (x - \hat{\mu}_B)^2$$

    - Adding $\epsilon > 0$ ensures numerical stability when computing the square root:

$$\hat{\sigma}_B = \sqrt{\hat{\sigma}_B^2 + \epsilon}$$

Step 2: Normalize the Inputs

Normalize the input x by subtracting the batch mean and dividing by the batch standard deviation:

$$\hat{x}_i = \frac{x_i - \hat{\mu}_B}{\hat{\sigma}_B}$$

- This step ensures the inputs have **zero mean** and **unit variance**.
- Intuition: This normalization ensures that features are on a similar scale, improving optimization stability.

Step 3: Apply Scaling and Shifting

The normalized inputs $\hat{x}$ are scaled by γ and shifted by β:

$$BN(x) = \gamma \odot \hat{x} + \beta$$

- γ (scale) and β (shift) are learnable parameters.
- Why scaling and shifting?
    - Without these parameters, normalization might overly constrain the model.
    - Allowing the model to learn γ and β ensures that the network can adapt to the optimal distribution for each feature.

Example

Let's assume a mini-batch $\mathcal{B}$ contains 3 samples for a single feature:

$$x = [10, 20, 30]$$

1. **Compute the Mean**:

$$\hat{\mu}_B = \frac{10 + 20 + 30}{3} = 20$$

2. **Compute the Variance**:

$$\hat{\sigma}_B^2 = \frac{(10 - 20)^2 + (20 - 20)^2 + (30 - 20)^2}{3} = \frac{100 + 0 + 100}{3} = 66.67$$

   Add $\epsilon = 10^{-5}$ (small value for stability).

3. **Standard Deviation**:

$$\hat{\sigma}_B = \sqrt{66.67 + 10^{-5}} \approx 8.16$$

4. **Normalize**:

$$\hat{x} = \frac{x - \hat{\mu}_B}{\hat{\sigma}_B} = \frac{[10, 20, 30] - 20}{8.16} = [-1.22, 0, 1.22]$$

5. **Apply Scaling and Shifting**:

   - Assume $\gamma = 2$ and $\beta = 1$:

$$BN(x) = \gamma \odot \hat{x} + \beta = 2 \cdot [-1.22, 0, 1.22] + 1 = [-1.44, 1, 3.44]$$

NOTE: these parameters are **learnable**, the network learns them during training in the same way as it learns the weights *w*

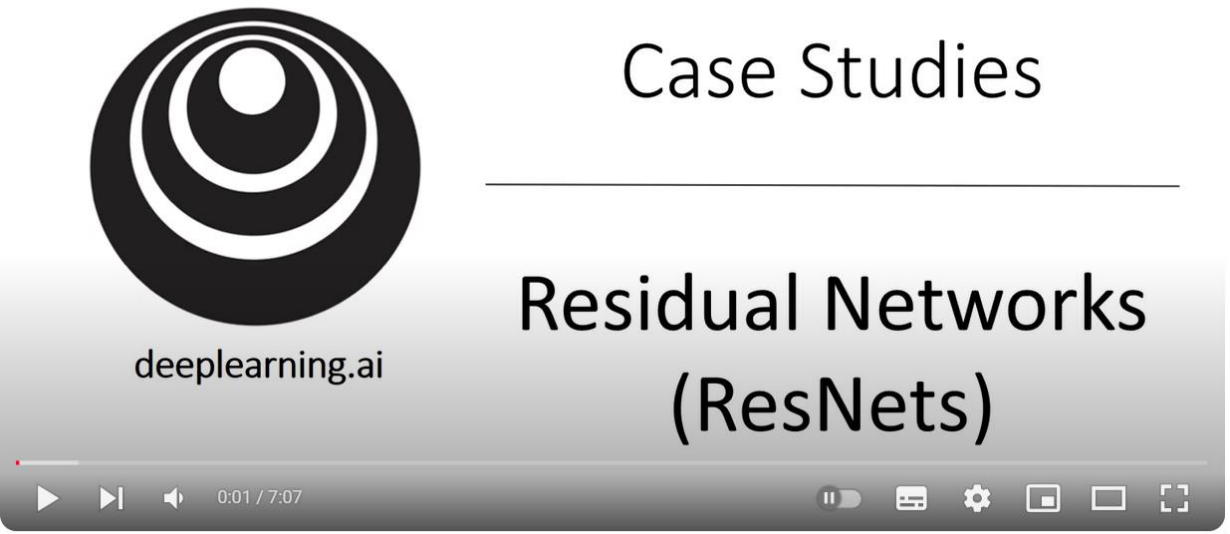
Code part

- we have 2 types of batch normalization in PyTorch:
  - ***nn.BatchNorm2d***: for batch normalization applied after convolutional layers
  - ***nn.BatchNorm1d***: for batch normalization applied after fully-connected layers

```
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5), nn.BatchNorm2d(6), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.BatchNorm2d(16), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2), nn.Flatten(),
    nn.Linear(256, 120), nn.BatchNorm1d(120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.BatchNorm1d(84), nn.Sigmoid(),
    nn.Linear(84, 10))
```

- the parameter of *BatchNorm* represents the number of output channels of the previous layer

9) ResNet



C4W2L03 Resnets

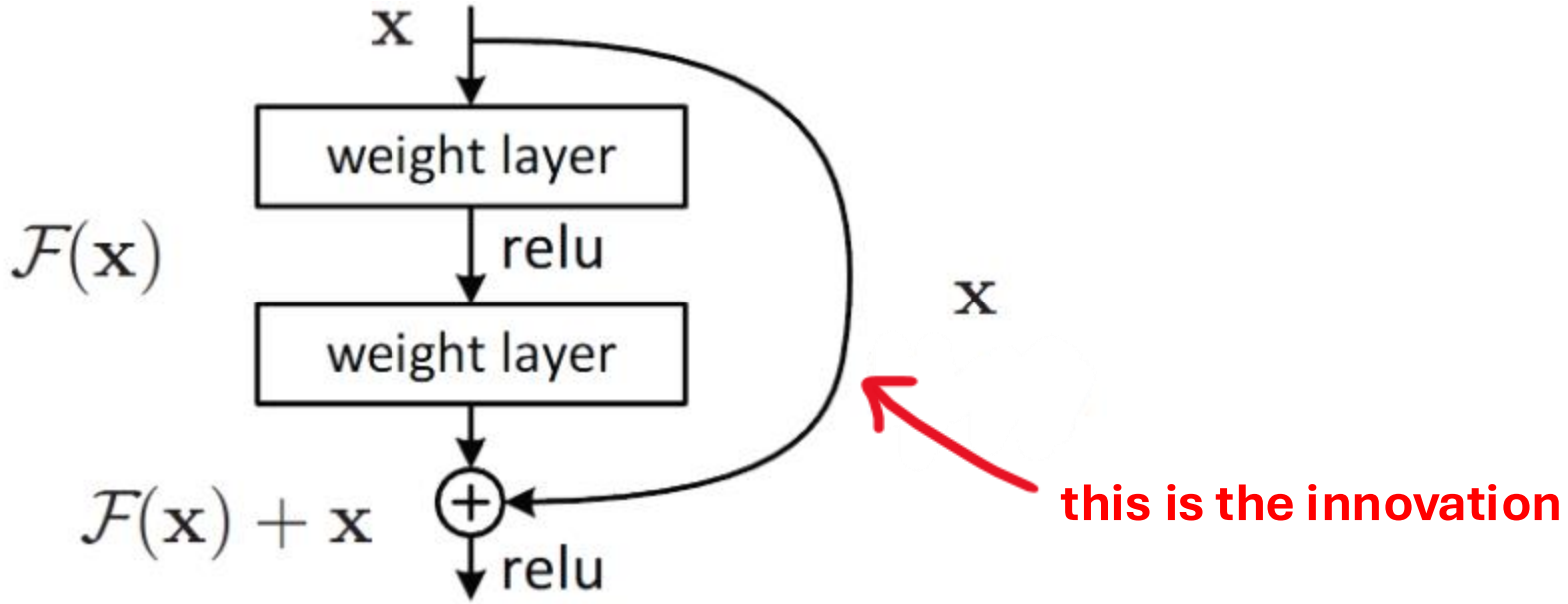


C4W2L04 Why ResNets Work

- introduced by Kaiming He and colleagues in 2015, is a groundbreaking DL architecture that addressed one of the major challenges in deep neural networks: the **degradation problem**

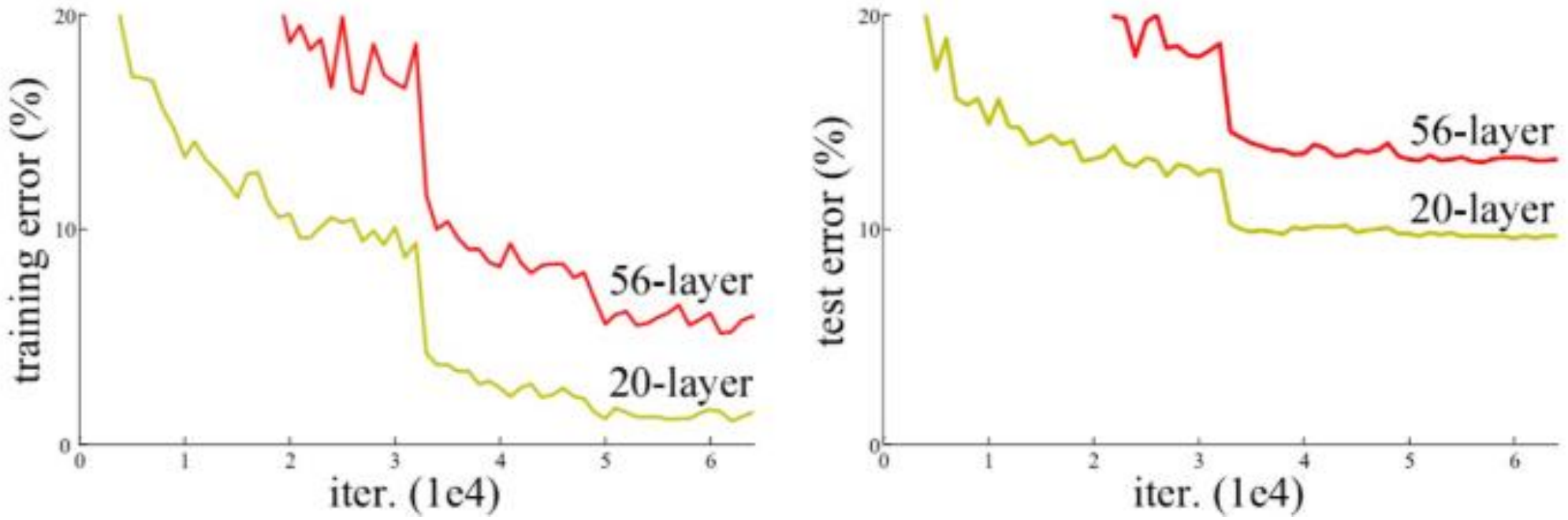


- ResNet became famous for winning the **ImageNet Large Scale Visual Recognition Challenge (ILSVRC) 2015** with a top-5 error rate of just **3.57%**, surpassing human-level performance

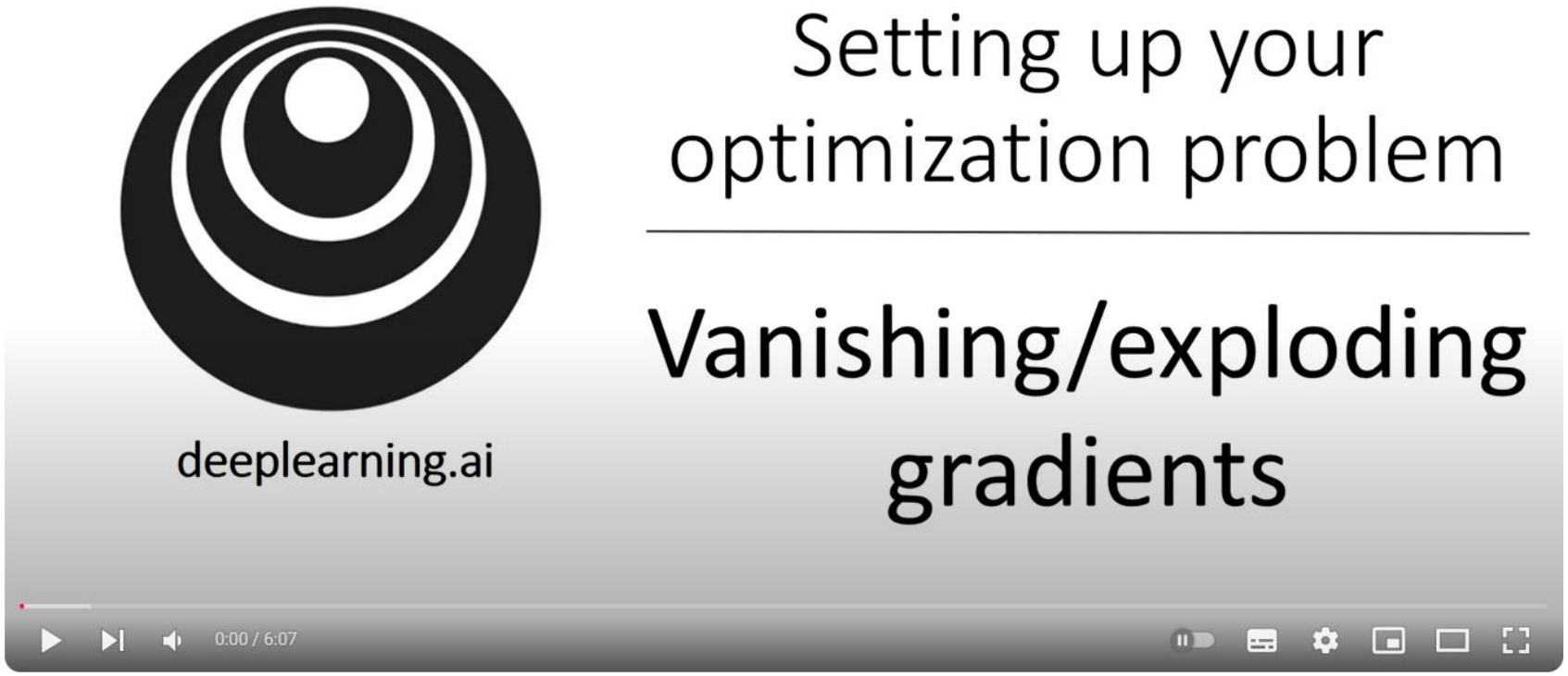


Degradation problem

- the **degradation problem** in deep learning refers to a phenomenon where increasing the depth of a neural network (i.e., adding more layers) leads to **worse performance** on the training dataset, despite the expectation that deeper networks should perform better



Watch this video to understand more



Vanishing/Exploding Gradients (C2W1L10)

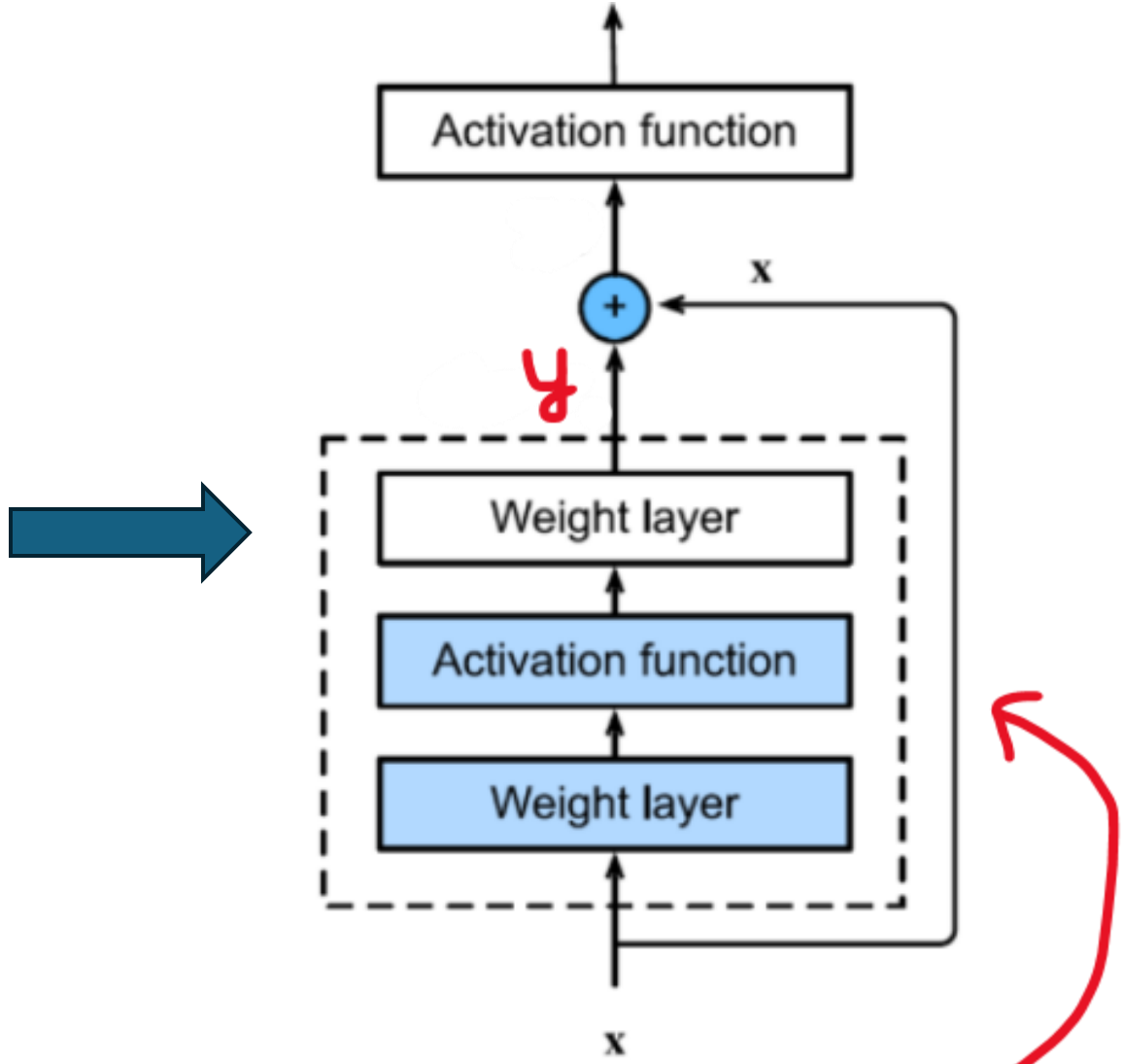
What makes ResNet so special?

- **solved the degradation problem**: when increasing the depth of a neural network, deeper models sometimes perform worse than shallower ones due to the difficulty in optimizing very deep networks. ResNet solved this by introducing **residual connections**

- ResNet demonstrated that extremely deep networks (e.g., 152 layers in ResNet-152) could be trained effectively

```python
class Residual(nn.Module):
    """The Residual block of ResNet."""
    def __init__(self, input_channels, num_channels,
                 use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels,
                               kernel_size=3, padding=1, stride=strides)
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.conv2 = nn.Conv2d(num_channels, num_channels,
                               kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(num_channels)
        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels,
                                   kernel_size=1, stride=strides)
        else:
            self.conv3 = None

    def forward(self, X):
        Y = nn.ReLU()(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return nn.ReLU()(Y)
```



this is the innovation

```
b1 = nn.Sequential(nn.Conv2d(1, 64, kernel_size=7, stride=2, padding=3),
                   nn.BatchNorm2d(64), nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))
```

1 layer here

```
def resnet_block(input_channels, num_channels, num_residuals,
                 first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
        else:
            blk.append(Residual(num_channels, num_channels))
    return blk
```

```
b2 = nn.Sequential(*resnet_block(64, 64, 2, first_block=True))
b3 = nn.Sequential(*resnet_block(64, 128, 2))
b4 = nn.Sequential(*resnet_block(128, 256, 2))
b5 = nn.Sequential(*resnet_block(256, 512, 2))
```

4 *resnet_block*, each containing 2 *Residual* and each *Residual* contains 2 layers (see previous slide – we don't count 1x1 conv) ➡ **16** layers

```
net = nn.Sequential(b1, b2, b3, b4, b5,
                    nn.AdaptiveAvgPool2d((1, 1)),
                    nn.Flatten(), nn.Linear(512, 10))
```

1 layer here

this is known as the famous ResNet-18 – because we have 18 layers

NOTE:

ResNet Architectures

ResNet comes in various configurations, defined by the number of layers:

- **ResNet-18**: 18 layers.
- **ResNet-34**: 34 layers.
- **ResNet-50**: 50 layers.
- **ResNet-101**: 101 layers.
- **ResNet-152**: 152 layers.

Old Exercises (same as for PI)

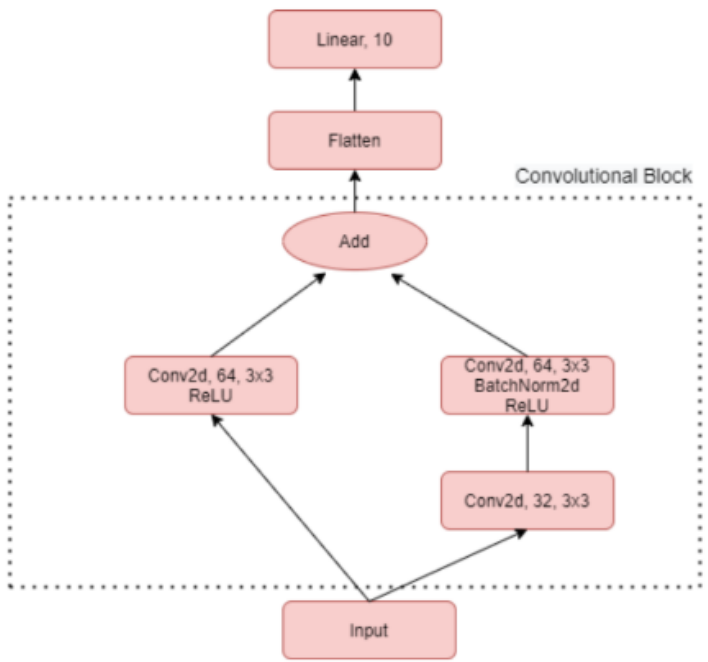
Laboratory 9 – Exercises

1. Perform the cross-correlation operation between an input tensor with 3 input channels and the following kernel:

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \begin{bmatrix} 7 & 2 \\ 3 & 5 \end{bmatrix}.$$

2. Classify the SVHN (Street View House Numbers) dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using the LeNet model. Divide the training dataset as follows: 30000 validation images and 43257 training images. Set the learning rate to 0.5.

3. Classify the SVHN dataset using the following convolutional neural network architecture. Divide the training dataset as follows: 30000 validation images and 43257 training images. Train your model for 5 epochs using a learning rate of 0.05 and a batch size of 256. Define a class for implementing the Convolutional Block. Conv2d, 32, 3×3 means 32 output channels and 3×3 kernel size. Use padding, if necessary.



Solutions

1. Perform the cross-correlation operation between an input tensor with 3 input channels and the following kernel:

$$\begin{bmatrix} 0 & 1 \\ 2 & 3 \end{bmatrix}, \begin{bmatrix} 1 & 2 \\ 3 & 4 \end{bmatrix}, \begin{bmatrix} 7 & 2 \\ 3 & 5 \end{bmatrix}.$$

you also have an example in lab ☺

```python
import torch
from torch import nn
from torch.nn import functional as F
import torchvision
from torchvision import transforms
import matplotlib.pyplot as plt

torch.manual_seed(42) # Set a fixed random seed for reproducibility
torch.backends.cudnn.deterministic = True # Ensure deterministic results for operations involving CUDA (not strictly necessary for CPU)

# Define the input tensor X - I choose these values for X just in case you want to compute the cross-correlation operation with pen and paper :)
# Otherwise you can choose random values for X
# X is a 4D tensor with shape (batch_size=1, channels=3, height=3, width=3):
#     -batch size is 1, indicating one input sample
#     -channels = 3 means the input has 3 input channels
#     -height and width = 3 each, representing a 3x3 spatial dimension
X = torch.tensor([[[[1.0, 1.0, 1.0], [1.0, 1.0, 1.0], [1.0, 1.0, 1.0]],
                   [[1.0, 1.0, 1.0], [1.0, 1.0, 1.0], [1.0, 1.0, 1.0]],
                   [[1.0, 1.0, 1.0], [1.0, 1.0, 1.0], [1.0, 1.0, 1.0]]]])

# Define the kernel tensor K
# K is a 4D tensor with shape (out_channels=1, in_channels=3, kernel_height=2, kernel_width=2):
#     -out_channels = 1, indicating the kernel produces one output feature map
#     -in_channels = 3, matching the input's 3 channels
#     -kernel size is 2x2
K = torch.tensor([[[[0.0, 1.0], [2.0, 3.0]], [[1.0, 2.0], [3.0, 4.0]],[[7.0, 2.0], [3.0, 5.0]]]])

# Perform the cross-correlation operation using 2D convolution (default stride is 1, and padding is 0)
F.conv2d(X, K)
```

```
tensor([[[[33., 33.],
          [33., 33.]]]])
```

2. Classify the SVHN (Street View House Numbers) dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using the LeNet model. Divide the training dataset as follows: 30000 validation images and 43257 training images. Set the learning rate to 0.5.

how did I know to change them? ☺ (see next slides)

```python
# Define the LeNet model architecture
net = nn.Sequential(
    nn.Conv2d(3, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 6 * 6, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))

# Verify the output shape at each layer
X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
for layer in net: # Iterate through each layer of the network
    X = layer(X) # Pass the input through the layer
    print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

```
Conv2d output shape:     torch.Size([1, 6, 32, 32])
Sigmoid output shape:    torch.Size([1, 6, 32, 32])
AvgPool2d output shape:  torch.Size([1, 6, 16, 16])
Conv2d output shape:     torch.Size([1, 16, 12, 12])
Sigmoid output shape:    torch.Size([1, 16, 12, 12])
AvgPool2d output shape:  torch.Size([1, 16, 6, 6])
Flatten output shape:    torch.Size([1, 576])
Linear output shape:     torch.Size([1, 120])
Sigmoid output shape:    torch.Size([1, 120])
Linear output shape:     torch.Size([1, 84])
Sigmoid output shape:    torch.Size([1, 84])
Linear output shape:     torch.Size([1, 10])
```

LeNet from the lab:

```
net = nn.Sequential(
    nn.Conv2d(1, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))
```

- now we are working with SVHN dataset which contains RGB images (3 channels) so we need to change from 1 channel to 3 channels



```
# Define the LeNet model architecture
net = nn.Sequential(
    nn.Conv2d(3, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))

# Verify the output shape at each layer
X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
for layer in net: # Iterate through each layer of the network
    X = layer(X) # Pass the input through the layer
    print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

```
Conv2d output shape:     torch.Size([1, 6, 32, 32])
Sigmoid output shape:    torch.Size([1, 6, 32, 32])
AvgPool2d output shape:  torch.Size([1, 6, 16, 16])
Conv2d output shape:     torch.Size([1, 16, 12, 12])
Sigmoid output shape:    torch.Size([1, 16, 12, 12])
AvgPool2d output shape:  torch.Size([1, 16, 6, 6])
Flatten output shape:    torch.Size([1, 576])
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
<ipython-input-5-04643f0b7056> in <cell line: 14>()
     13 X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
     14 for layer in net: # Iterate through each layer of the network
---> 15     X = layer(X) # Pass the input through the layer
     16     print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

- next if we run the code in this form we will have an error; Why? ☺

```python
# Define the LeNet model architecture
net = nn.Sequential(
    nn.Conv2d(3, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 5 * 5, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))

# Verify the output shape at each layer
X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
for layer in net: # Iterate through each layer of the network
    X = layer(X) # Pass the input through the layer
    print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

```
Conv2d output shape:     torch.Size([1, 6, 32, 32])
Sigmoid output shape:    torch.Size([1, 6, 32, 32])
AvgPool2d output shape:  torch.Size([1, 6, 16, 16])
Conv2d output shape:     torch.Size([1, 16, 12, 12])
Sigmoid output shape:    torch.Size([1, 16, 12, 12])
AvgPool2d output shape:  torch.Size([1, 16, 6, 6])
Flatten output shape:    torch.Size([1, 576])
---------------------------------------------------------------------------
RuntimeError                              Traceback (most recent call last)
<ipython-input-5-04643f0b7056> in <cell line: 14>()
     13 X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
     14 for layer in net: # Iterate through each layer of the network
---> 15     X = layer(X) # Pass the input through the layer
     16     print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

- originally, LeNet was developed to work with single-channel (black and white) **28x28** images
- now we are working with SVHN dataset which contains RGB images of **32x32**, so we need to change the number of input neurons for the first *nn.Linear* after *nn.Flatten*



```python
# Define the LeNet model architecture
net = nn.Sequential(
    nn.Conv2d(3, 6, kernel_size=5, padding=2), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Conv2d(6, 16, kernel_size=5), nn.Sigmoid(),
    nn.AvgPool2d(kernel_size=2, stride=2),
    nn.Flatten(),
    nn.Linear(16 * 6 * 6, 120), nn.Sigmoid(),
    nn.Linear(120, 84), nn.Sigmoid(),
    nn.Linear(84, 10))

# Verify the output shape at each layer
X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
for layer in net: # Iterate through each layer of the network
    X = layer(X) # Pass the input through the layer
    print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

```
Conv2d output shape:     torch.Size([1, 6, 32, 32])
Sigmoid output shape:    torch.Size([1, 6, 32, 32])
AvgPool2d output shape:  torch.Size([1, 6, 16, 16])
Conv2d output shape:     torch.Size([1, 16, 12, 12])
Sigmoid output shape:    torch.Size([1, 16, 12, 12])
AvgPool2d output shape:  torch.Size([1, 16, 6, 6])
Flatten output shape:    torch.Size([1, 576])
Linear output shape:     torch.Size([1, 120])
Sigmoid output shape:    torch.Size([1, 120])
Linear output shape:     torch.Size([1, 84])
Sigmoid output shape:    torch.Size([1, 84])
Linear output shape:     torch.Size([1, 10])
```

- use functions from this lab:

adaptation from *load_data_fashion_mnist()* function

```python
# Function to load and preprocess the SVHN dataset
def load_data_SVHN(batch_size, resize=None):
    """Download the SVHN dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.SVHN(root="../data", split='train', transform=trans, download=True)
    mnist_test = torchvision.datasets.SVHN(root="../data", split='test', transform=trans, download=True)
    # Split the training dataset into training (43257 samples) and validation (30000 samples) sets
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [43257, 30000], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))
```

I changed these

```python
# Function to evaluate the model's accuracy and loss on a given dataset
def evaluate_accuracy(net, data_iter, loss, device):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to train the model for one epoch
def train_epoch(net, train_iter, loss, optimizer, device):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        X, y = X.to(device), y.to(device)
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to select the device (GPU if available, otherwise CPU)
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')
```

```python
# Function to train the model over multiple epochs
def train(net, train_iter, val_iter, test_iter, num_epochs, lr, device):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    def init_weights(m):
        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('Training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer, device)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter, loss, device)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')
    test_loss, test_acc = evaluate_accuracy(net, test_iter, loss, device)
    print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')

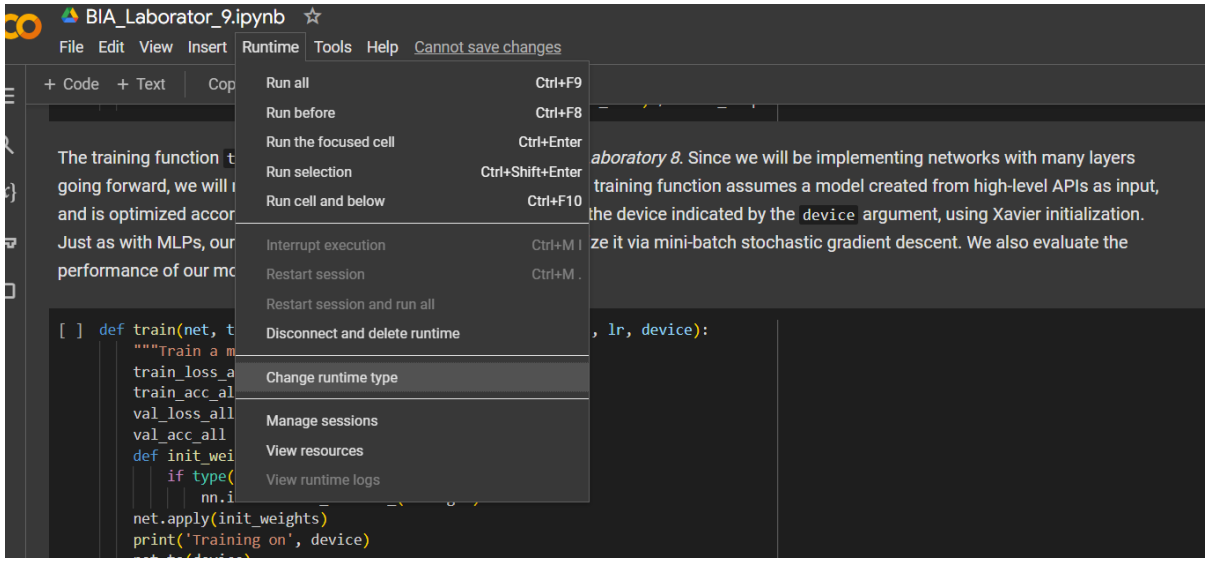
    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```

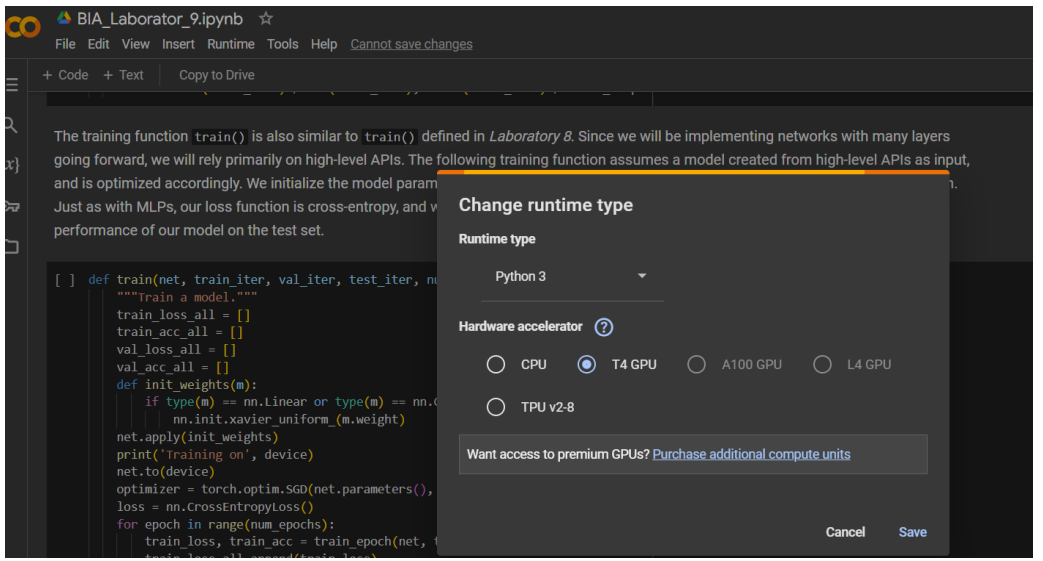
```python
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()
```

```python
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()
```

NOTE: don't forget to change the runtime from CPU to GPU:

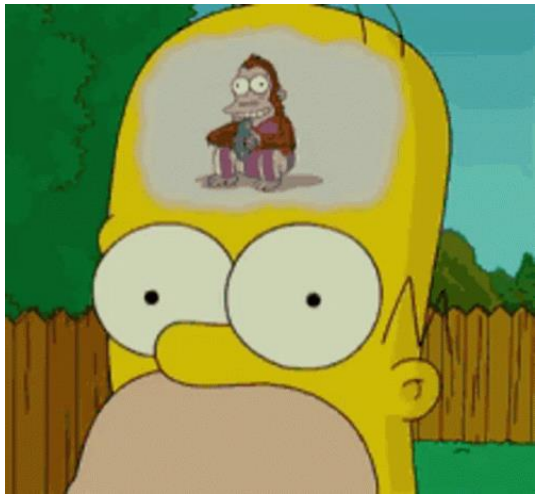




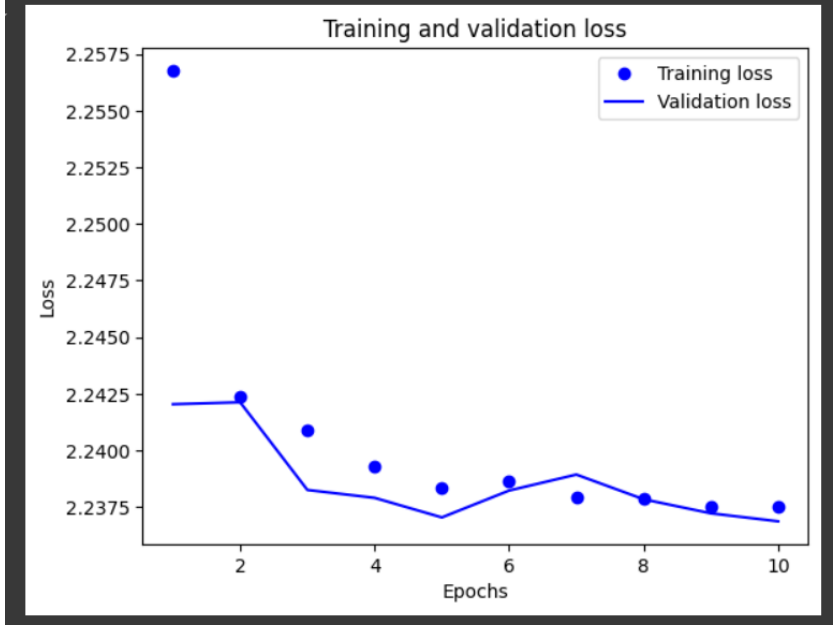


```python
# Set hyperparameters for training
batch_size, lr, num_epochs = 256, 0.5, 10 # Batch size = 256, learning rate = 0.5, epochs = 10
train_iter, val_iter, test_iter = load_data_SVHN(batch_size) # Load training, validation, and test datasets
# Train the model and return loss/accuracy history
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, test_iter, num_epochs, lr, try_gpu())
```

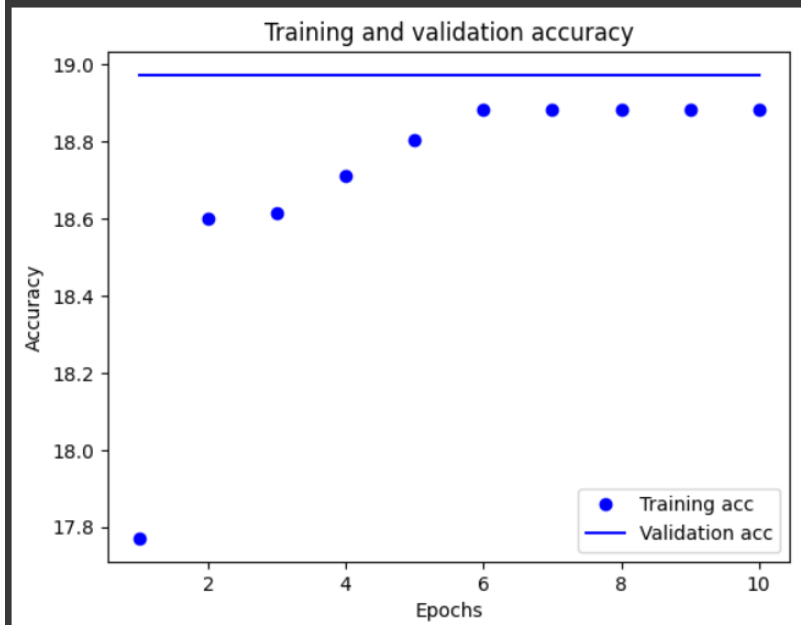
```
Downloading http://ufldl.stanford.edu/housenumbers/train_32x32.mat to ../data/train_32x32.mat
100%|██████████| 182M/182M [00:24<00:00, 7.41MB/s]
Downloading http://ufldl.stanford.edu/housenumbers/test_32x32.mat to ../data/test_32x32.mat
100%|██████████| 64.3M/64.3M [00:13<00:00, 4.67MB/s]
Training on cuda:0
Epoch 1, Train loss 2.26, Train accuracy 17.77, Validation loss 2.24, Validation accuracy 18.97
Epoch 2, Train loss 2.24, Train accuracy 18.60, Validation loss 2.24, Validation accuracy 18.97
Epoch 3, Train loss 2.24, Train accuracy 18.61, Validation loss 2.24, Validation accuracy 18.97
Epoch 4, Train loss 2.24, Train accuracy 18.71, Validation loss 2.24, Validation accuracy 18.97
Epoch 5, Train loss 2.24, Train accuracy 18.80, Validation loss 2.24, Validation accuracy 18.97
Epoch 6, Train loss 2.24, Train accuracy 18.88, Validation loss 2.24, Validation accuracy 18.97
Epoch 7, Train loss 2.24, Train accuracy 18.88, Validation loss 2.24, Validation accuracy 18.97
Epoch 8, Train loss 2.24, Train accuracy 18.88, Validation loss 2.24, Validation accuracy 18.97
Epoch 9, Train loss 2.24, Train accuracy 18.88, Validation loss 2.24, Validation accuracy 18.97
Epoch 10, Train loss 2.24, Train accuracy 18.88, Validation loss 2.24, Validation accuracy 18.97
Test loss 2.23, Test accuracy 19.59
```



```python
plot_loss(train_loss_all, val_loss_all)
```




```python
plot_accuracy(train_acc_all, val_acc_all)
```



NOTE: the model is too dumb to learn this task, this is why we have so bad performance ☺

Optional

adaptation from *get_fashion_mnist_labels()* function from lab 8

```python
def get_svhn_labels(labels):
    """Return text labels for the SVHN dataset."""
    text_labels = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
    return [text_labels[int(i)] for i in labels]
```

```python
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```

function from lab 8

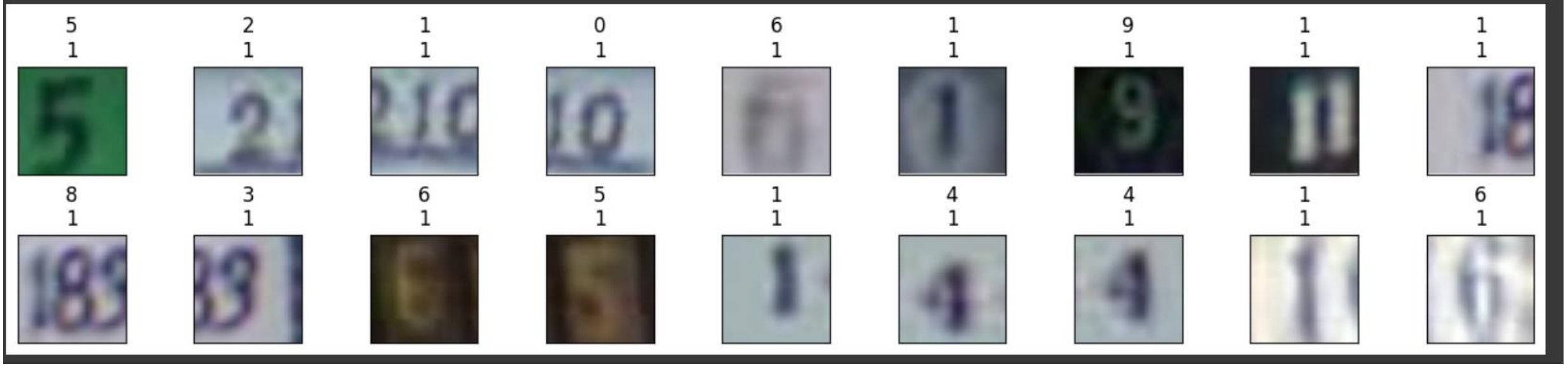
Optional

- use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

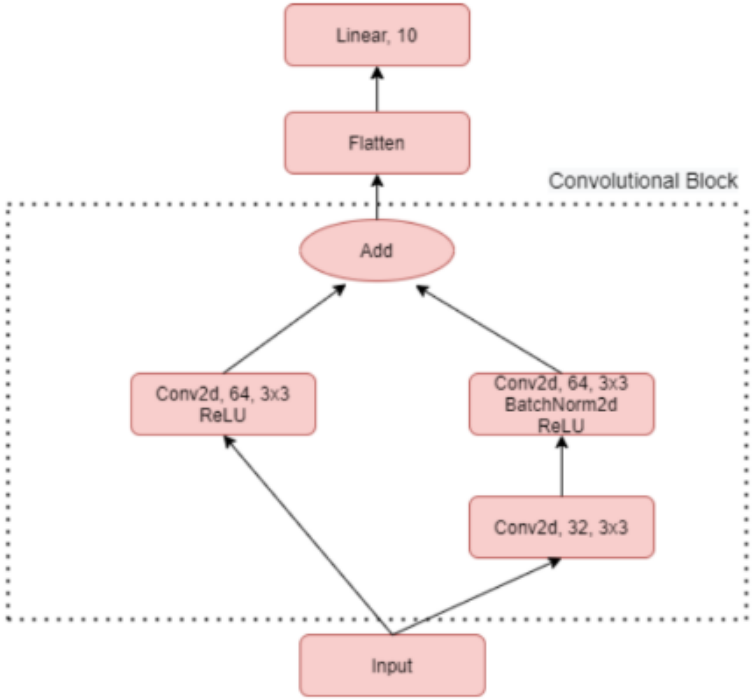
```
def predict(net, test_iter, n=18):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_svhn_labels(y)
    preds = get_svhn_labels(net(X.cuda()).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].permute(0, 2, 3, 1), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```

I needed to change these 2 things



3. Classify the SVHN dataset using the following convolutional neural network architecture. Divide the training dataset as follows: 30000 validation images and 43257 training images. Train your model for 5 epochs using a learning rate of 0.05 and a batch size of 256. Define a class for implementing the Convolutional Block. Conv2d, 32, 3×3 means 32 output channels and 3×3 kernel size. Use padding, if necessary.




```python
class ConvBlock(nn.Module):

    # Constructor to initialize layers in the block
    def __init__(self):
        super().__init__()
        # First convolutional layer: 3 input channels, 64 output channels, kernel size 3x3
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3)
        # Second convolutional layer: 3 input channels, 32 output channels, kernel size 3x3
        self.conv2 = nn.Conv2d(3,32,kernel_size=3)
        # Third convolutional layer: 32 input channels, 64 output channels, kernel size 3x3, padding=1
        # Padding ensures that the spatial dimensions of the output match the input !
        self.conv3 = nn.Conv2d(32,64,kernel_size=3,padding=1)
        # Batch normalization layer for the third convolutional output
        # Normalizes the feature maps across the batch to improve training stability
        self.bn = nn.BatchNorm2d(64)

    # Forward method to define how the input passes through the block
    def forward(self,X):
      # First branch: Pass input through conv1 and apply ReLU activation
        y1 = nn.ReLU()(self.conv1(X))
        # Second branch: Pass input through conv2
        y2 = self.conv2(X)
        # Pass the output of conv2 (y2) through conv3, apply batch normalization, and then ReLU activation
        y3 = nn.ReLU()(self.bn(self.conv3(y2)))
        # Add the outputs of the first branch (y1) and the third branch (y3)
        # This follows the skip-connection pattern (like in ResNet)
        return y1 + y3

# Instantiate the ConvBlock
conv = ConvBlock()
# Define the full network by adding the ConvBlock, a flattening layer, and a final linear layer
# Flatten layer reshapes the feature maps into a vector for the fully connected layer
# Linear layer maps the vector to 10 output classes (for SVHN classification)
net = nn.Sequential(conv,nn.Flatten(),nn.Linear(57600,10))

X = torch.randn(1, 3, 32, 32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
ConvBlock output shape:  torch.Size([1, 64, 30, 30])
Flatten output shape:    torch.Size([1, 57600])
Linear output shape:     torch.Size([1, 10])
```

NOTES

- the input image (from SVHN dataset) is a color image, so I set the number of input channels for *self.conv1* and *self.conv2* to **3**
- how did I know to add *padding=1* for *self.conv3*?
- let's do it without *padding=1*:

```python
class ConvBlock(nn.Module):

    # Constructor to initialize layers in the block
    def __init__(self):
        super().__init__()
        # First convolutional layer: 3 input channels, 64 output channels, kernel size 3x3
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3)
        # Second convolutional layer: 3 input channels, 32 output channels, kernel size 3x3
        self.conv2 = nn.Conv2d(3,32,kernel_size=3)
        # Third convolutional layer: 32 input channels, 64 output channels, kernel size 3x3, padding=1
        # Padding ensures that the spatial dimensions of the output match the input !
        self.conv3 = nn.Conv2d(32,64,kernel_size=3,padding=1)
        # Batch normalization layer for the third convolutional output
        # Normalizes the feature maps across the batch to improve training stability
        self.bn = nn.BatchNorm2d(64)

    # Forward method to define how the input passes through the block
    def forward(self,X):
        # First branch: Pass input through conv1 and apply ReLU activation
        y1 = nn.ReLU()(self.conv1(X))
        # Second branch: Pass input through conv2
        y2 = self.conv2(X)
        # Pass the output of conv2 (y2) through conv3, apply batch normalization, and then ReLU activation
        y3 = nn.ReLU()(self.bn(self.conv3(y2)))
        # Add the outputs of the first branch (y1) and the third branch (y3)
        # This follows the skip-connection pattern (like in ResNet)
        return y1 + y3

# Instantiate the ConvBlock
conv = ConvBlock()
# Define the full network by adding the ConvBlock, a flattening layer, and a final linear layer
# Flatten layer reshapes the feature maps into a vector for the fully connected layer
# Linear layer maps the vector to 10 output classes (for SVHN classification)
net = nn.Sequential(conv,nn.Flatten(),nn.Linear(57600,10))

X = torch.randn(1, 3, 32, 32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

aici e buba !

```
RuntimeError                              Traceback (most recent call last)
<ipython-input-25-e104d9c8b33c> in <cell line: 37>()
     36 X = torch.randn(1, 3, 32, 32)
     37 for layer in net:
---> 38     X = layer(X)
     39     print(layer.__class__.__name__, 'output shape:\t', X.shape)

2 frames
<ipython-input-25-e104d9c8b33c> in forward(self, X)
     25         # Add the outputs of the first branch (y1) and the third branch (y3)
     26         # This follows the skip-connection pattern (like in ResNet)
---> 27         return y1 + y3
     28
     29 # Instantiate the ConvBlock

RuntimeError: The size of tensor a (30) must match the size of tensor b (28) at non-singleton dimension 3
```

- let's print the shapes to debug:

```
    # Forward method to define how the input passes through the block
    def forward(self,X):
        # First branch: Pass input through conv1 and apply ReLU activation
        y1 = nn.ReLU()(self.conv1(X))
        # Second branch: Pass input through conv2
        y2 = self.conv2(X)
        # Pass the output of conv2 (y2) through conv3, apply batch normalization, and then ReLU activation
        y3 = nn.ReLU()(self.bn(self.conv3(y2)))
        # Add the outputs of the first branch (y1) and the third branch (y3)
        # This follows the skip-connection pattern (like in ResNet)
        print(y1.shape)
        print(y2.shape)
        print(y3.shape)
        return y1 + y3

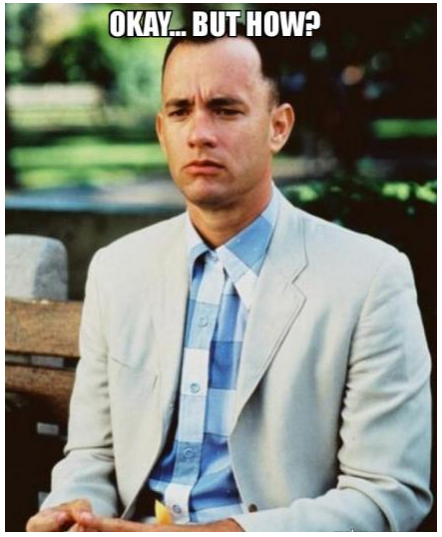
# Instantiate the ConvBlock
conv = ConvBlock()
# Define the full network by adding the ConvBlock, a flattening layer, and a final linear layer
# Flatten layer reshapes the feature maps into a vector for the fully connected layer
# Linear layer maps the vector to 10 output classes (for SVHN classification)
net = nn.Sequential(conv,nn.Flatten(),nn.Linear(57600,10))

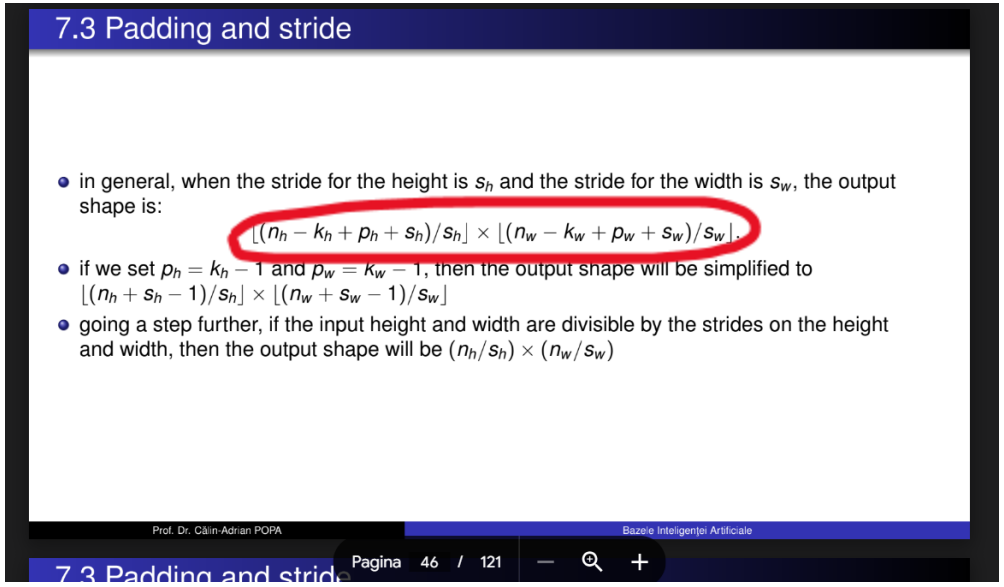
X = torch.randn(1, 3, 32, 32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
torch.Size([1, 64, 30, 30])
torch.Size([1, 32, 30, 30])
torch.Size([1, 64, 28, 28])
```

y3 has the (spatial) shape 28x28 and
y1 has the (spatial) shape 30x30 !

we need to add some *padding* or *stride* to self.conv3 to change the spatial shape of y3 from 28x28 to 30x30





memorize this slide for the exam ☺

- in our case:
  - $n_h = n_w = 30$ (because y2 has the spatial shape 30x30
  - $k_h = k_w = 3$ (because the *kernel_size* = *3* for self.conv3)
  - $p_h = p_w$ (because by default *padding* = *0* for torch.nn.Conv2d) → **if we change this to 1 will solve the problem**
  - $s_h = s_w = 0$ (because by default *stride* = *1* for torch.nn.Conv2d)

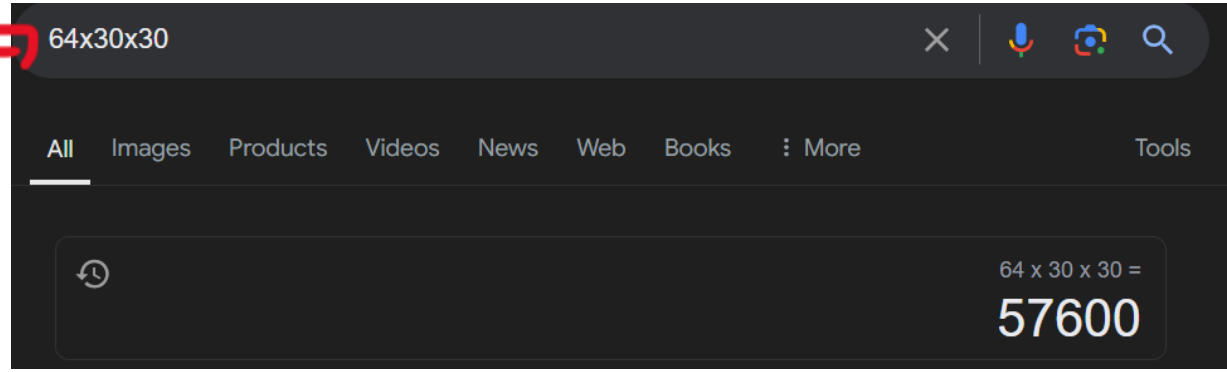
NOTE: if *padding* = p, PyTorch applies p pixels of padding to both sides of the input (top and bottom, left and right). Therefore, the total padding added to the input along one dimension is $2 \times p$, so if I set *padding*=1, the p_h and p_w from the formula from the course will be 2

- ok, next how did I know to set this number?

```
# Instantiate the ConvBlock
conv = ConvBlock()
# Define the full network by adding the ConvBlock, a flattening layer, and a final linear layer
# Flatten layer reshapes the feature maps into a vector for the fully connected layer
# Linear layer maps the vector to 10 output classes (for SVHN classification)
net = nn.Sequential(conv,nn.Flatten(),nn.Linear(57600,10))
```

```python
X = torch.randn(1, 3, 32, 32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
ConvBlock output shape:  torch.Size([1, 64, 30, 30])
Flatten output shape:    torch.Size([1, 57600])
Linear output shape:     torch.Size([1, 10])
```





- use functions from this lab:

adaptation from *load_data_fashion_mnist()* function

```python
# Function to load and preprocess the SVHN dataset
def load_data_SVHN(batch_size, resize=None):
    """Download the SVHN dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.SVHN(root="../data", split='train', transform=trans, download=True)
    mnist_test = torchvision.datasets.SVHN(root="../data", split='test', transform=trans, download=True)
    # Split the training dataset into training (43257 samples) and validation (30000 samples) sets
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [43257, 30000], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))
```

```python
# Function to evaluate the model's accuracy and loss on a given dataset
def evaluate_accuracy(net, data_iter, loss, device):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to train the model for one epoch
def train_epoch(net, train_iter, loss, optimizer, device):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        X, y = X.to(device), y.to(device)
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to select the device (GPU if available, otherwise CPU)
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')
```

```python
# Function to train the model over multiple epochs
def train(net, train_iter, val_iter, test_iter, num_epochs, lr, device):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    def init_weights(m):
        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('Training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer, device)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter, loss, device)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')
    test_loss, test_acc = evaluate_accuracy(net, test_iter, loss, device)
    print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')

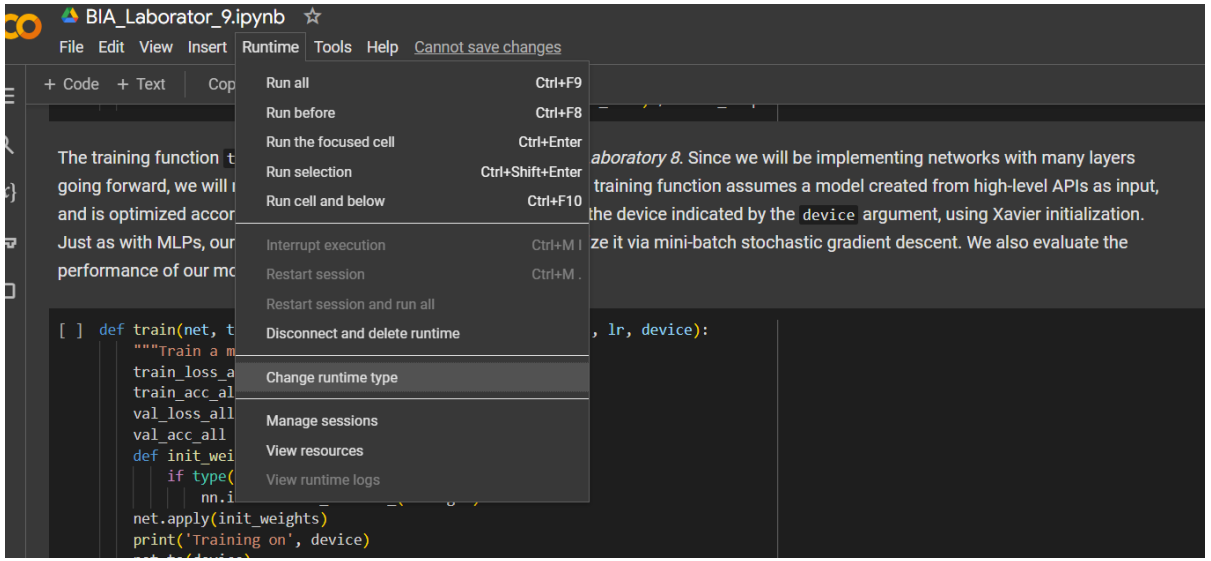
    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```

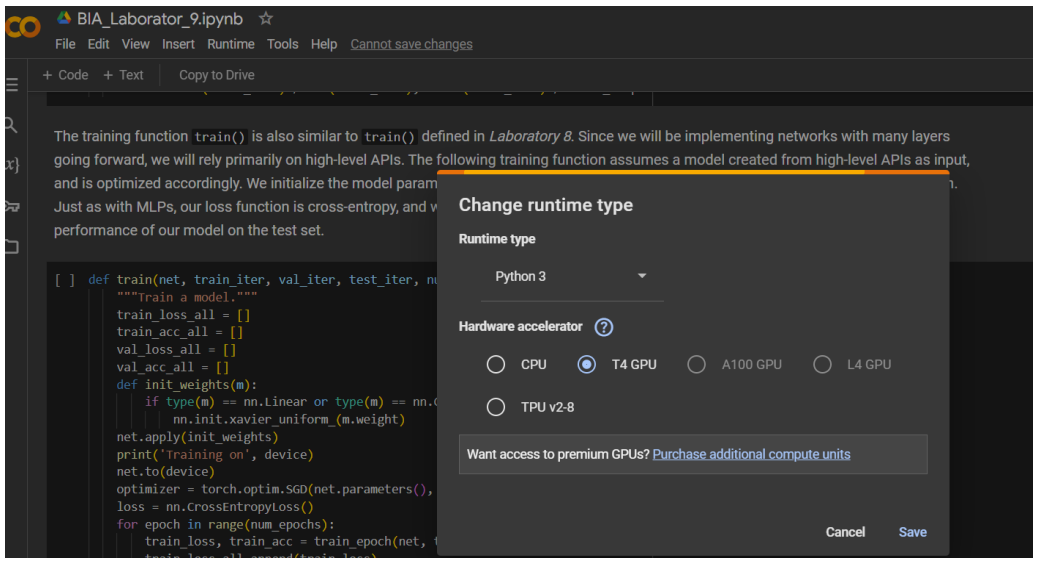
```python
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()
```

```python
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()
```

NOTE: don't forget to change the runtime from CPU to GPU:



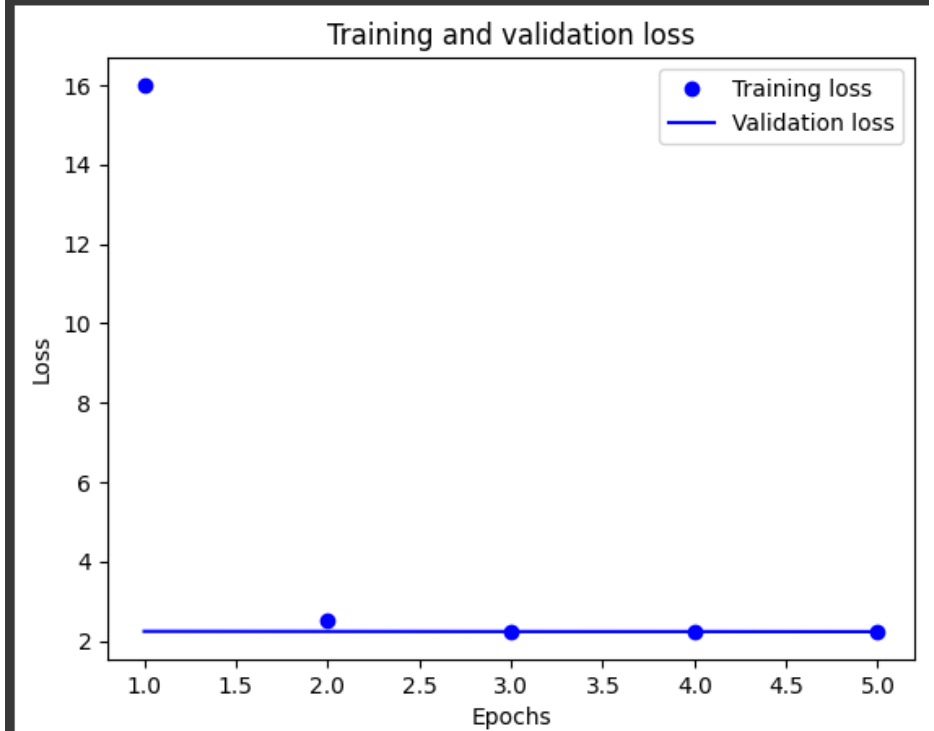




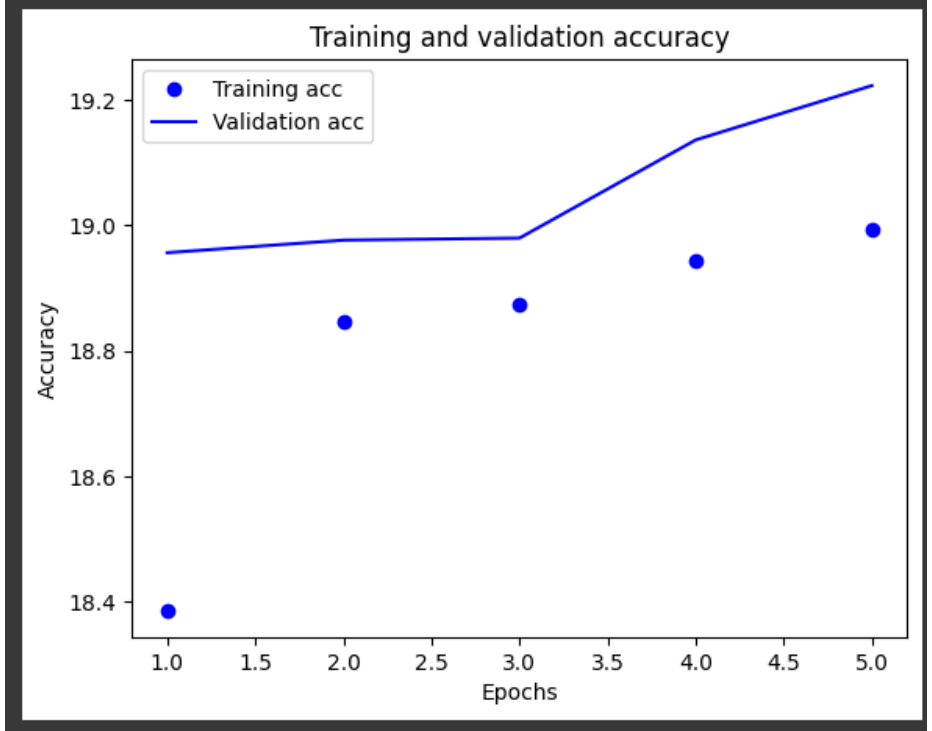
```
# Set hyperparameters for training
batch_size, lr, num_epochs = 256, 0.5, 5 # Batch size = 256, learning rate = 0.5, epochs = 10
train_iter, val_iter, test_iter = load_data_SVHN(batch_size) # Load training, validation, and test datasets
# Train the model and return loss/accuracy history
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, test_iter, num_epochs, lr, try_gpu())
```

```
Using downloaded and verified file: ../data/train_32x32.mat
Using downloaded and verified file: ../data/test_32x32.mat
Training on cuda:0
Epoch 1, Train loss 16.00, Train accuracy 18.39, Validation loss 2.24, Validation accuracy 18.96
Epoch 2, Train loss 2.52, Train accuracy 18.85, Validation loss 2.24, Validation accuracy 18.98
Epoch 3, Train loss 2.23, Train accuracy 18.87, Validation loss 2.23, Validation accuracy 18.98
Epoch 4, Train loss 2.23, Train accuracy 18.94, Validation loss 2.23, Validation accuracy 19.14
Epoch 5, Train loss 2.23, Train accuracy 18.99, Validation loss 2.23, Validation accuracy 19.22
Test loss 2.22, Test accuracy 19.91
```

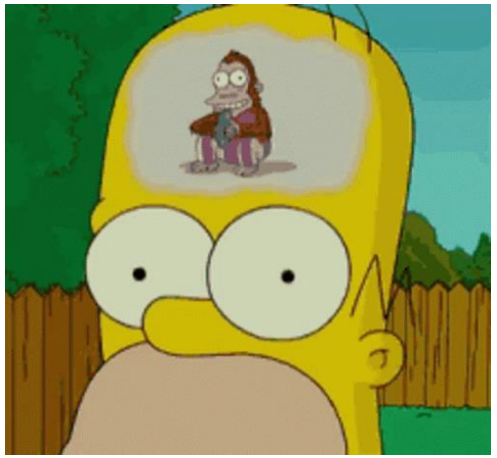
```
plot_loss(train_loss_all, val_loss_all)
```




```
plot_accuracy(train_acc_all, val_acc_all)
```



NOTE: the model is too dumb to learn this task, this is why we have so bad performance ☺



Optional

adaptation from *get_fashion_mnist_labels()* function from lab 8

```python
def get_svhn_labels(labels):
    """Return text labels for the SVHN dataset."""
    text_labels = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
    return [text_labels[int(i)] for i in labels]
```

```python
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```

function from lab 8

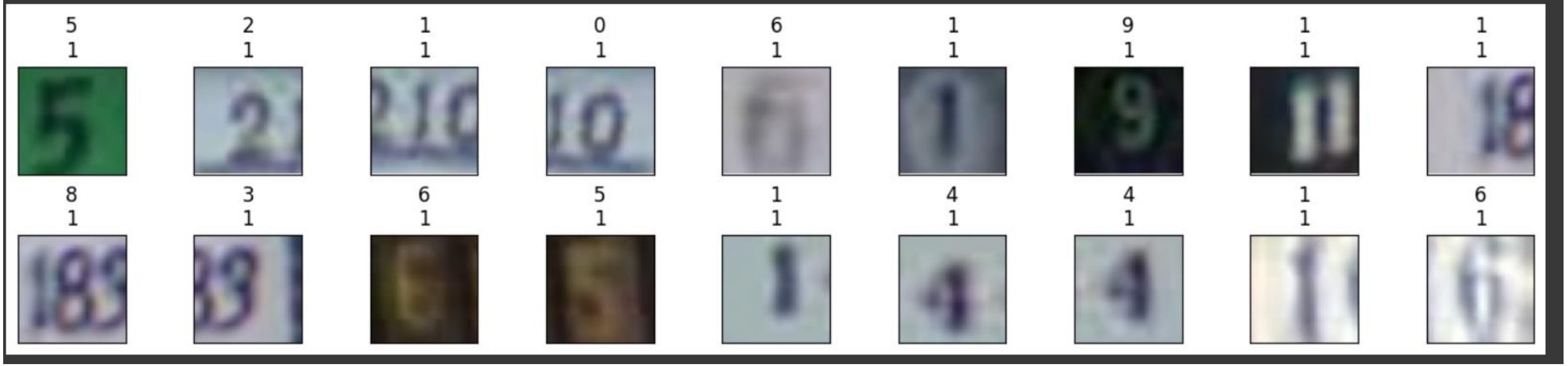
Optional

- use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

```
def predict(net, test_iter, n=18):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_svhn_labels(y)
    preds = get_svhn_labels(net(X.cuda()).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].permute(0, 2, 3, 1), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```

I needed to change these 2 things



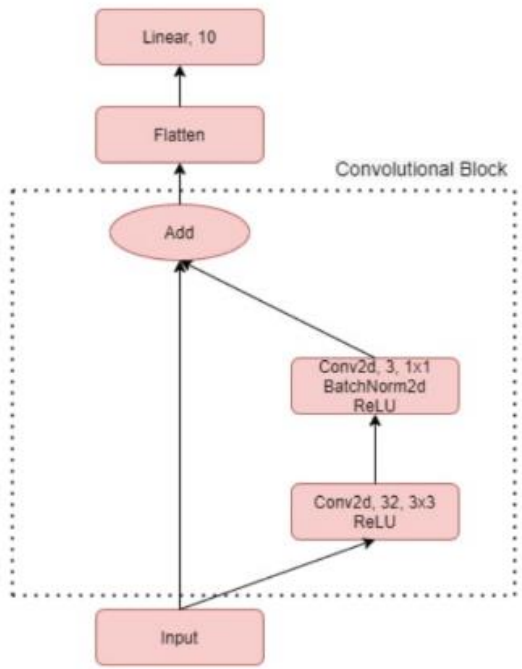
New Exercises

Laboratory 9 – Exercises

1. Define an input tensor of shape (1,1,7,7) with random values. The two-dimensional convolutional layer should have as arguments: $c_in = 1$, $c_out = 1$, $padding = (2, 1)$ and $stride = (3, 3)$. Set the $kernel_size$ argument such that the output shape of the convolutional operation to be (1,1,3,2).

2. Classify the SVHN dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using ResNet architecture by applying only two modules of residual blocks. Train the model for 10 epochs with $lr = 0.9$ and $batch_size = 256$. Plot the training and validation accuracy.

3. Classify the SVHN dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using the following convolutional neural network architecture. Divide the training dataset as follows: 30000 validation images and 43257 training images. Train your model for 5 epochs using a learning rate of 0.05 and a batch size of 256. Define a class for implementing the Convolutional Block. Conv2d, 32, 3×3 means 32 output channels and 3×3 kernel size. Use padding, if necessary.



Solutions

1. Define an input tensor of shape (1,1,7,7) with random values. The two-dimensional convolutional layer should have as arguments: $c_in = 1$, $c_out = 1$, $padding = (2, 1)$ and $stride = (3, 3)$. Set the $kernel_size$ argument such that the output shape of the convolutional operation to be (1,1,3,2).

```python
import torch
from torch import nn
from torch.nn import functional as F
import torchvision
from torchvision import transforms
import matplotlib.pyplot as plt

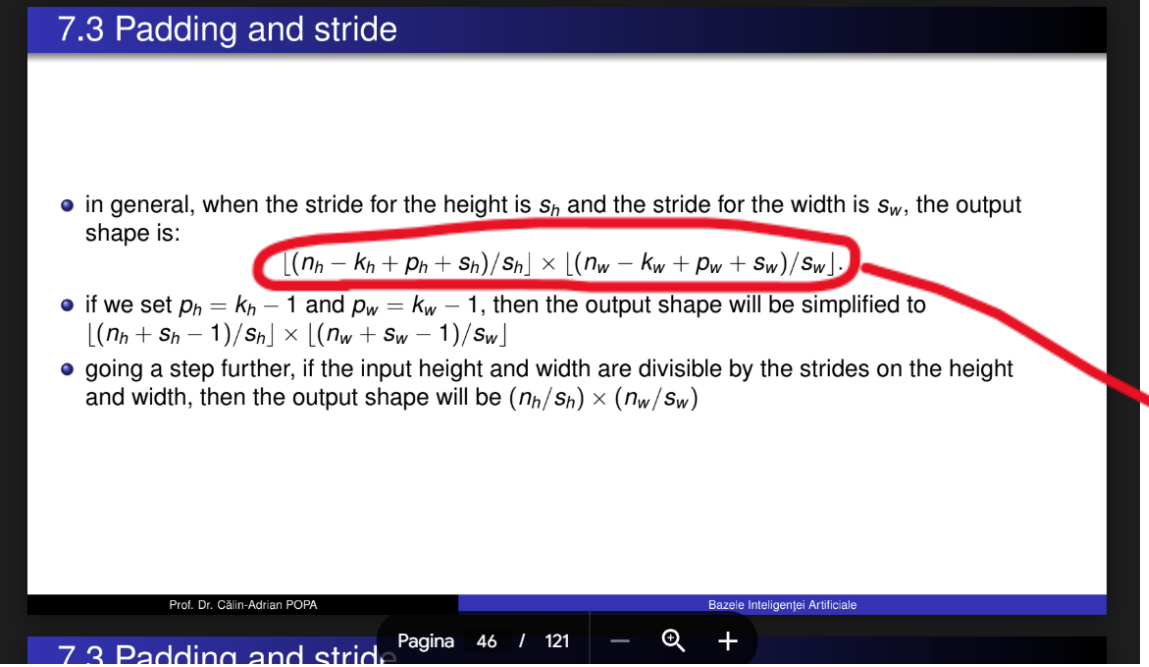
torch.manual_seed(42) # Set a fixed random seed for reproducibility
torch.backends.cudnn.deterministic = True # Ensure deterministic results for operations involving CUDA (not strictly necessary for CPU)

# Define the input tensor and convolutional layer
input_tensor = torch.randn(1, 1, 7, 7)  # Random input of shape (1, 1, 7, 7)
conv_layer = nn.Conv2d(
    in_channels=1,
    out_channels=1,
    kernel_size=(3, 5),  # Derived kernel size
    stride=(3, 3),
    padding=(2, 1),
)
output_tensor = conv_layer(input_tensor)
print("Exercise 1 Output Shape:", output_tensor.shape)  # Should be (1, 1, 3, 2)
```

How did I know that? ☺



```
Exercise 1 Output Shape: torch.Size([1, 1, 3, 2])
```



memorize this slide for the exam ☺

- $\lfloor (7 - k_h + 2\times2 + 3) / 3 \rfloor = 3 \Rightarrow k_h = 3$
- $\lfloor (7 - k_w + 2\times1 + 3) / 3 \rfloor = 2 \Rightarrow k_w = 5$

because the output shape needs to be (1, 1, 3, 2)

- in our case:
  - $n_h = n_w = 7$
  - $k_h = ?$, $k_w = ?$
  - $p_h = 2$, $p_w = 1$
  - $s_h = s_w = 3$

NOTE: if *padding* = p, PyTorch applies p pixels of padding to both sides of the input (top and bottom, left and right). Therefore, the total padding added to the input along one dimension is $2 \times p$, so if I set *padding*=(2, 1), the p_h and p_w from the formula from the course will be 2×2 and 2×1

2. Classify the SVHN dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using ResNet architecture by applying only two modules of residual blocks. Train the model for 10 epochs with $lr = 0.9$ and $batch_size = 256$. Plot the training and validation accuracy.

```python
class Residual(nn.Module):
    """The Residual block of ResNet."""
    def __init__(self, input_channels, num_channels,
                 use_1x1conv=False, strides=1):
        super().__init__()
        self.conv1 = nn.Conv2d(input_channels, num_channels,
                               kernel_size=3, padding=1, stride=strides)
        self.bn1 = nn.BatchNorm2d(num_channels)
        self.conv2 = nn.Conv2d(num_channels, num_channels,
                               kernel_size=3, padding=1)
        self.bn2 = nn.BatchNorm2d(num_channels)
        if use_1x1conv:
            self.conv3 = nn.Conv2d(input_channels, num_channels,
                                   kernel_size=1, stride=strides)
        else:
            self.conv3 = None

    def forward(self, X):
        Y = nn.ReLU()(self.bn1(self.conv1(X)))
        Y = self.bn2(self.conv2(Y))
        if self.conv3:
            X = self.conv3(X)
        Y += X
        return nn.ReLU()(Y)
```

```python
def resnet_block(input_channels, num_channels, num_residuals,
                 first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
        else:
            blk.append(Residual(num_channels, num_channels))
    return blk


# Define the initial convolutional block (b1)
b1 = nn.Sequential(nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),
                   nn.BatchNorm2d(64), nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

# Residual blocks (already defined in the original code)
b2 = nn.Sequential(*resnet_block(64, 64, 2, first_block=True))  # First residual block
b3 = nn.Sequential(*resnet_block(64, 128, 2))  # Second residual block

net = nn.Sequential(b1, b2, b3,
                    nn.AdaptiveAvgPool2d((1, 1)),
                    nn.Flatten(), nn.Linear(128, 10))
net
```

```python
X = torch.rand(1, 3, 32, 32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
Sequential output shape:         torch.Size([1, 64, 8, 8])
Sequential output shape:         torch.Size([1, 64, 8, 8])
Sequential output shape:         torch.Size([1, 128, 4, 4])
AdaptiveAvgPool2d output shape:  torch.Size([1, 128, 1, 1])
Flatten output shape:    torch.Size([1, 128])
Linear output shape:     torch.Size([1, 10])
```

- the input image (from SVHN dataset) is a color image, so I set the number of input channels for *b1* to **3**

- then I applied only 2 modules of residual blocks, as specified in requirements

- finally, I changed the number of input neurons for the first *nn.Linear* after *nn.Flatten:*

```
def resnet_block(input_channels, num_channels, num_residuals,
                 first_block=False):
    blk = []
    for i in range(num_residuals):
        if i == 0 and not first_block:
            blk.append(Residual(input_channels, num_channels,
                                use_1x1conv=True, strides=2))
        else:
            blk.append(Residual(num_channels, num_channels))
    return blk


# Define the initial convolutional block (b1)
b1 = nn.Sequential(nn.Conv2d(3, 64, kernel_size=7, stride=2, padding=3),
                   nn.BatchNorm2d(64), nn.ReLU(),
                   nn.MaxPool2d(kernel_size=3, stride=2, padding=1))

# Residual blocks (already defined in the original code)
b2 = nn.Sequential(*resnet_block(64, 64, 2, first_block=True))  # First residual block
b3 = nn.Sequential(*resnet_block(64, 128, 2))  # Second residual block

net = nn.Sequential(b1, b2, b3,
                    nn.AdaptiveAvgPool2d((1, 1)),
                    nn.Flatten(), nn.Linear(128, 10))
net
```

```
X = torch.rand(1, 3, 32, 32)
for layer in net:
    X = layer(X)
    print(layer.__class__.__name__, 'output shape:\t', X.shape)
```

```
Sequential output shape:         torch.Size([1, 64, 8, 8])
Sequential output shape:         torch.Size([1, 64, 8, 8])
Sequential output shape:         torch.Size([1, 128, 4, 4])
AdaptiveAvgPool2d output shape:  torch.Size([1, 128, 1, 1])
Flatten output shape:    torch.Size([1, 128])
Linear output shape:     torch.Size([1, 10])
```

I looked here

- use functions from this lab:

adaptation from *load_data_fashion_mnist()* function

```python
# Function to load and preprocess the SVHN dataset
def load_data_SVHN(batch_size, resize=None):
    """Download the SVHN dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.SVHN(root="../data", split='train', transform=trans, download=True)
    mnist_test = torchvision.datasets.SVHN(root="../data", split='test', transform=trans, download=True)
    # Split the training dataset into training (43257 samples) and validation (30000 samples) sets
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [43257, 30000], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))
```

```python
# Function to evaluate the model's accuracy and loss on a given dataset
def evaluate_accuracy(net, data_iter, loss, device):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to train the model for one epoch
def train_epoch(net, train_iter, loss, optimizer, device):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        X, y = X.to(device), y.to(device)
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to select the device (GPU if available, otherwise CPU)
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')
```

```python
# Function to train the model over multiple epochs
def train(net, train_iter, val_iter, test_iter, num_epochs, lr, device):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    def init_weights(m):
        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('Training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer, device)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter, loss, device)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')
    test_loss, test_acc = evaluate_accuracy(net, test_iter, loss, device)
    print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')

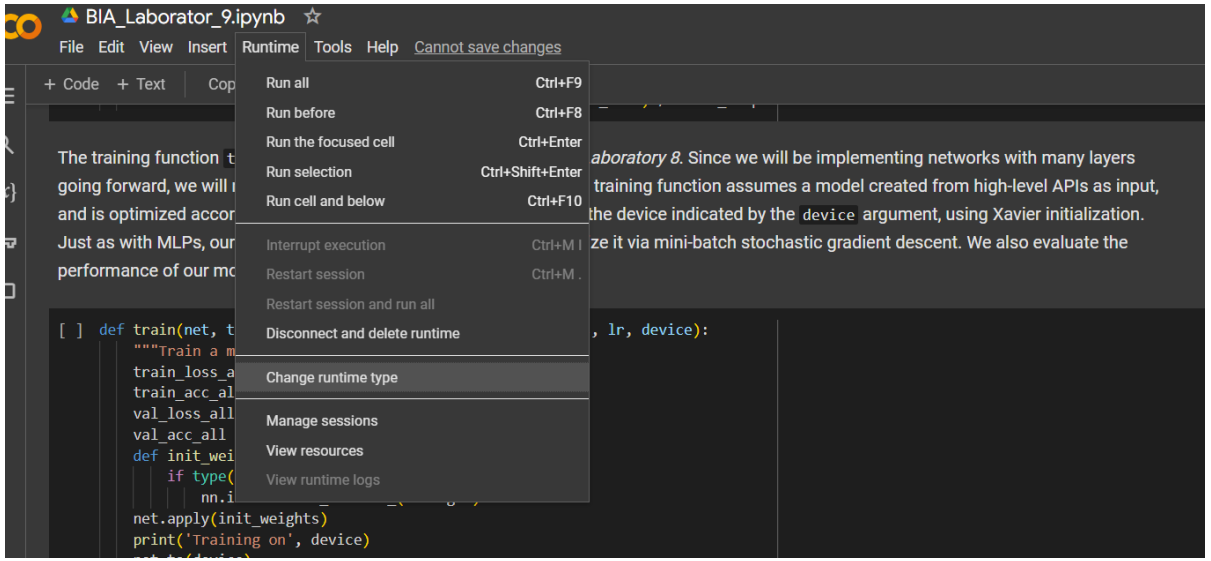
    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```

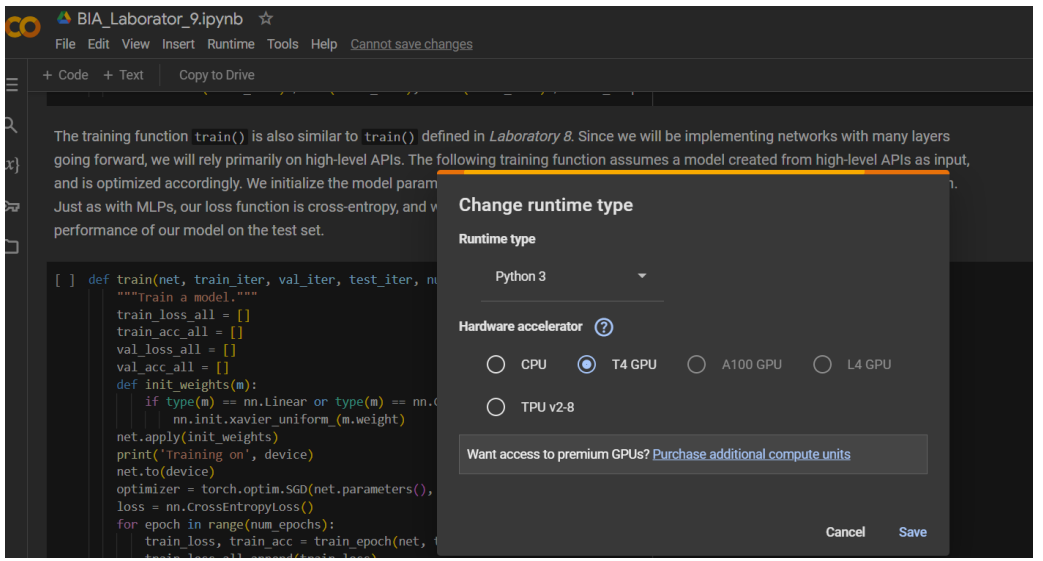
```python
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()
```

```python
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()
```

NOTE: don't forget to change the runtime from CPU to GPU:



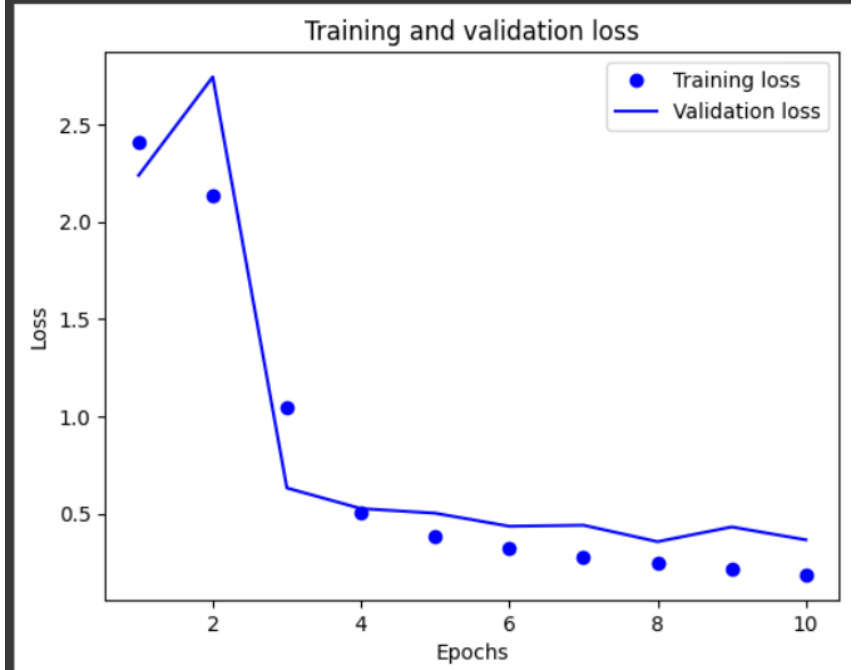




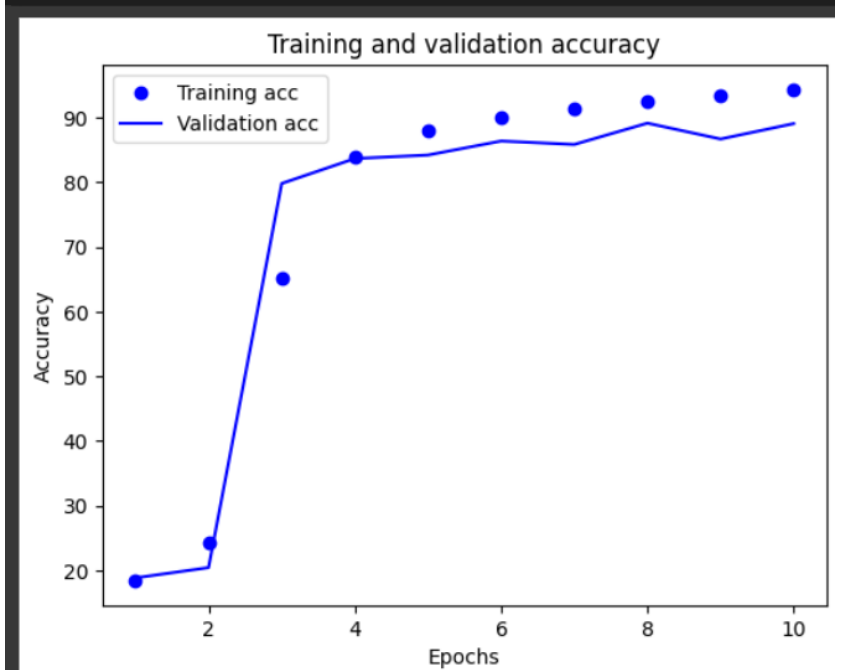
```python
# Set hyperparameters for training
batch_size, lr, num_epochs = 256, 0.9, 10
train_iter, val_iter, test_iter = load_data_SVHN(batch_size) # Load training, validation, and test datasets
# Train the model and return loss/accuracy history
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, test_iter, num_epochs, lr, try_gpu())
```

```
Using downloaded and verified file: ../data/train_32x32.mat
Using downloaded and verified file: ../data/test_32x32.mat
Training on cuda:0
Epoch 1, Train loss 2.41, Train accuracy 18.42, Validation loss 2.24, Validation accuracy 18.84
Epoch 2, Train loss 2.13, Train accuracy 24.38, Validation loss 2.74, Validation accuracy 20.42
Epoch 3, Train loss 1.05, Train accuracy 65.12, Validation loss 0.63, Validation accuracy 79.80
Epoch 4, Train loss 0.51, Train accuracy 83.92, Validation loss 0.53, Validation accuracy 83.66
Epoch 5, Train loss 0.39, Train accuracy 87.93, Validation loss 0.50, Validation accuracy 84.20
Epoch 6, Train loss 0.33, Train accuracy 90.00, Validation loss 0.44, Validation accuracy 86.36
Epoch 7, Train loss 0.28, Train accuracy 91.45, Validation loss 0.44, Validation accuracy 85.83
Epoch 8, Train loss 0.24, Train accuracy 92.57, Validation loss 0.36, Validation accuracy 89.13
Epoch 9, Train loss 0.22, Train accuracy 93.48, Validation loss 0.43, Validation accuracy 86.68
Epoch 10, Train loss 0.19, Train accuracy 94.32, Validation loss 0.37, Validation accuracy 89.06
Test loss 0.38, Test accuracy 88.81
```

```python
plot_loss(train_loss_all, val_loss_all)
```




```python
plot_accuracy(train_acc_all, val_acc_all)
```



Optional

adaptation from *get_fashion_mnist_labels()* function from lab 8

```python
def get_svhn_labels(labels):
    """Return text labels for the SVHN dataset."""
    text_labels = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
    return [text_labels[int(i)] for i in labels]
```

```python
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```

function from lab 8

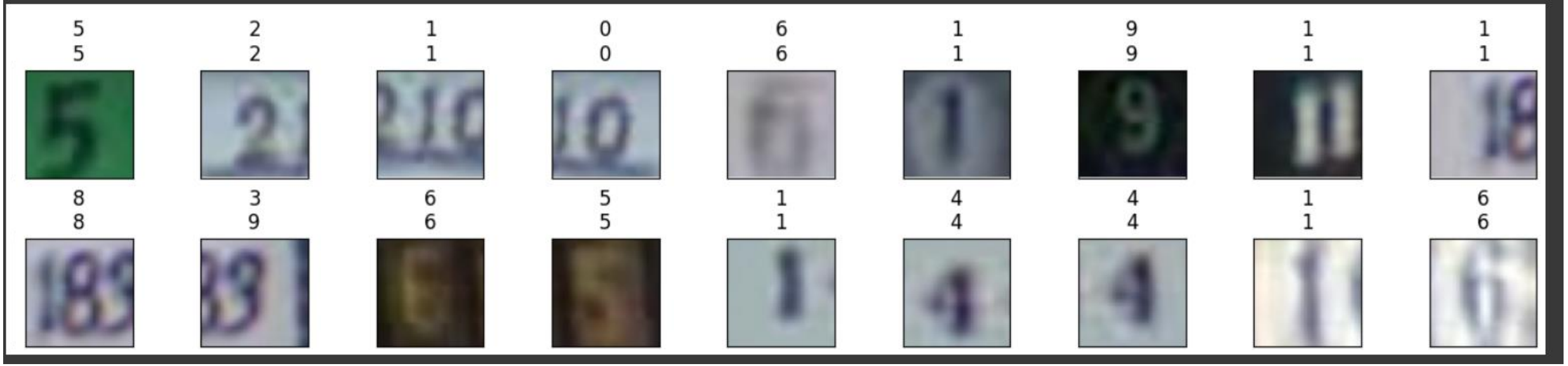
Optional

- use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

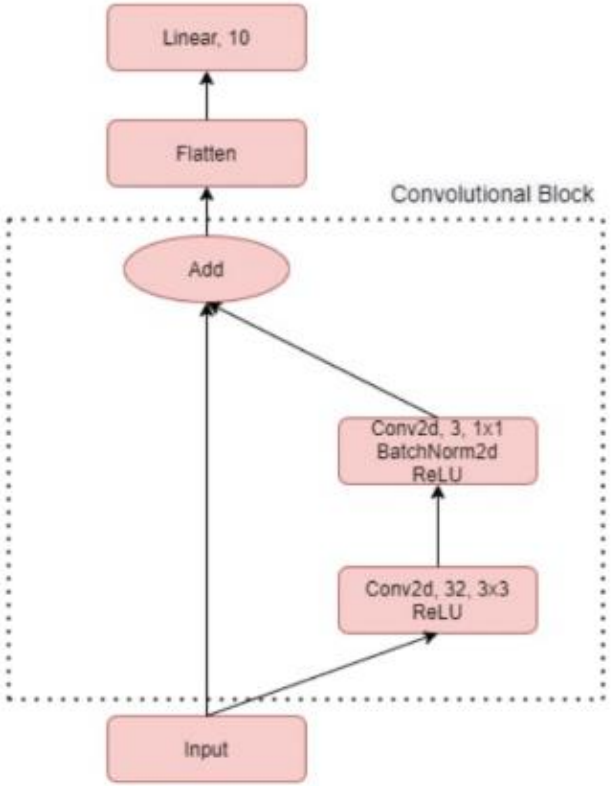
```
def predict(net, test_iter, n=18):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_svhn_labels(y)
    preds = get_svhn_labels(net(X.cuda()).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].permute(0, 2, 3, 1), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```

I needed to change these 2 things



3. Classify the SVHN dataset (32×32 images, 10 classes, 73257 training images and 26032 testing images) using the following convolutional neural network architecture. Divide the training dataset as follows: 30000 validation images and 43257 training images. Train your model for 5 epochs using a learning rate of 0.05 and a batch size of 256. Define a class for implementing the Convolutional Block. Conv2d, 32, 3×3 means 32 output channels and 3×3 kernel size. Use padding, if necessary.




```python
class ConvolutionalBlock(nn.Module):
    def __init__(self, in_channels):
        super(ConvolutionalBlock, self).__init__()
        # First convolutional layer (3x3 kernel with padding to preserve input dimensions)
        self.conv3x3 = nn.Conv2d(in_channels, 32, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        # Second convolutional layer (1x1 kernel)
        self.conv1x1 = nn.Conv2d(32, 3, kernel_size=1)
        self.bn = nn.BatchNorm2d(3)

    def forward(self, x):
        """
        Defines the forward pass through the ConvolutionalBlock.

        Args:
            x (Tensor): Input tensor of shape (batch_size, in_channels, height, width).

        Returns:
            Tensor: Output tensor of the same shape as the input.
        """

        # Branch 1: input x
        branch1 = x

        # Branch 2: Conv2d -> ReLU -> Conv2d -> BatchNorm -> ReLU
        branch2 = self.conv3x3(x)
        branch2 = self.relu(branch2)
        branch2 = self.conv1x1(branch2)
        branch2 = self.bn(branch2)
        branch2 = self.relu(branch2)

        # Combine the two branches by element-wise addition
        out = branch1 + branch2
        return out

# Define the complete model
net = nn.Sequential(
            ConvolutionalBlock(3),  # Convolutional block with 3 input channels
            nn.Flatten(),  # Flatten the output of the convolutional block into a 1D vector
            nn.Linear(3 * 32 * 32, 10)  # Fully connected layer to classify into 10 classes
        )
```

```python
# Verify the output shape at each layer
X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
for layer in net: # Iterate through each layer of the network
    X = layer(X) # Pass the input through the layer
    print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

```
ConvolutionalBlock output shape:         torch.Size([1, 3, 32, 32])
Flatten output shape:    torch.Size([1, 3072])
Linear output shape:     torch.Size([1, 10])
```

NOTES

- the input image (from SVHN dataset) is a color image, so I set the number of input channels for *self.conv3x3* to **3**
- how did I know to add *padding=1* for *self.conv3x3*?
- let's do it without *padding=1*:

```python
class ConvolutionalBlock(nn.Module):
    def __init__(self, in_channels):
        super(ConvolutionalBlock, self).__init__()
        # First convolutional layer (3x3 kernel with padding to preserve input dimensions)
        self.conv3x3 = nn.Conv2d(in_channels, 32, kernel_size=3, padding=1)
        self.relu = nn.ReLU()
        # Second convolutional layer (1x1 kernel)
        self.conv1x1 = nn.Conv2d(32, 3, kernel_size=1)
        self.bn = nn.BatchNorm2d(3)
```

```python
# Define the complete model
net = nn.Sequential(
            ConvolutionalBlock(3),  # Convolutional block with 3 input channels
            nn.Flatten(),  # Flatten the output of the convolutional block into a 1D vector
            nn.Linear(3 * 32 * 32, 10)  # Fully connected layer to classify into 10 classes
        )
```

```python
class ConvolutionalBlock(nn.Module):
    def __init__(self, in_channels):
        super(ConvolutionalBlock, self).__init__()
        # First convolutional layer (3x3 kernel with padding to preserve input dimensions)
        self.conv3x3 = nn.Conv2d(in_channels, 32, kernel_size=3)
        self.relu = nn.ReLU()
        # Second convolutional layer (1x1 kernel)
        self.conv1x1 = nn.Conv2d(32, 3, kernel_size=1)
        self.bn = nn.BatchNorm2d(3)

    def forward(self, x):
        """
        Defines the forward pass through the ConvolutionalBlock.

        Args:
            x (Tensor): Input tensor of shape (batch_size, in_channels, height, width).

        Returns:
            Tensor: Output tensor of the same shape as the input.
        """

        # Branch 1: input x
        branch1 = x

        # Branch 2: Conv2d -> ReLU -> Conv2d -> BatchNorm -> ReLU
        branch2 = self.conv3x3(x)
        branch2 = self.relu(branch2)
        branch2 = self.conv1x1(branch2)
        branch2 = self.bn(branch2)
        branch2 = self.relu(branch2)

        # Combine the two branches by element-wise addition
        out = branch1 + branch2
        return out
```

aici e buba !

```python
# Verify the output shape at each layer
X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
for layer in net: # Iterate through each layer of the network
    X = layer(X) # Pass the input through the layer
    print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape
```

```
RuntimeError                              Traceback (most recent call last)
<ipython-input-29-4d1e734392aa> in <cell line: 3>()
      2 X = torch.randn(1, 3, 32, 32) # Randomly generate a single input sample (batch size = 1, 3 channels, 32x32 image)
      3 for layer in net: # Iterate through each layer of the network
----> 4     X = layer(X) # Pass the input through the layer
      5     print(layer.__class__.__name__, 'output shape:\t', X.shape) # Print the class name of the layer and the output shape

                                  2 frames
<ipython-input-28-32a34e512bd8> in forward(self, x)
     31
     32         # Combine the two branches by element-wise addition
---> 33         out = branch1 + branch2
     34         return out
     35

RuntimeError: The size of tensor a (32) must match the size of tensor b (30) at non-singleton dimension 3
```

7.3 Padding and stride

- in general, when the stride for the height is s_h and the stride for the width is s_w, the output shape is:

$$\lfloor (n_h - k_h + p_h + s_h)/s_h \rfloor \times \lfloor (n_w - k_w + p_w + s_w)/s_w \rfloor.$$

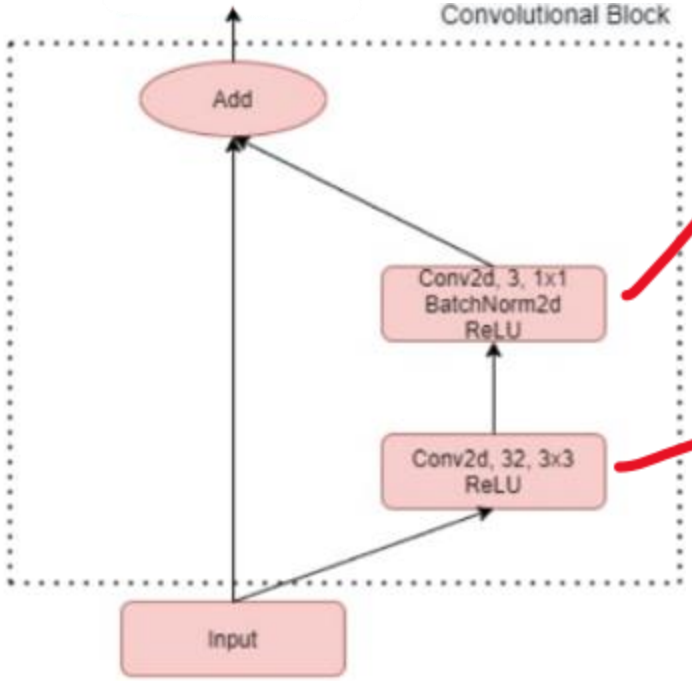
- if we set $p_h = k_h - 1$ and $p_w = k_w - 1$, then the output shape will be simplified to $\lfloor (n_h + s_h - 1)/s_h \rfloor \times \lfloor (n_w + s_w - 1)/s_w \rfloor$
- going a step further, if the input height and width are divisible by the strides on the height and width, then the output shape will be $(n_h/s_h) \times (n_w/s_w)$

Prof. Dr. Călin-Adrian POPA — Bazele Inteligenței Artificiale

Pagina 46 / 121

memorize this slide for the exam ☺

$\lfloor (n - 1 + 0 + 1) / 1 \rfloor = n$ (because by default *padding* = *0* and *stride* = *1*) ⇒ the 1x1 conv leave the input shape unchanged !



we need to look at this conv to see what padding should we add; after applying the formula, *padding* = *1* is a good choice



- use functions from this lab:

adaptation from *load_data_fashion_mnist()* function

```python
# Function to load and preprocess the SVHN dataset
def load_data_SVHN(batch_size, resize=None):
    """Download the SVHN dataset and then load it into memory."""
    trans = [transforms.ToTensor()]
    if resize:
        trans.insert(0, transforms.Resize(resize))
    trans = transforms.Compose(trans)
    mnist_train = torchvision.datasets.SVHN(root="../data", split='train', transform=trans, download=True)
    mnist_test = torchvision.datasets.SVHN(root="../data", split='test', transform=trans, download=True)
    # Split the training dataset into training (43257 samples) and validation (30000 samples) sets
    mnist_train, mnist_val = torch.utils.data.random_split(mnist_train, [43257, 30000], generator=torch.Generator().manual_seed(42))
    return (torch.utils.data.DataLoader(mnist_train, batch_size, shuffle=True, num_workers=2),
            torch.utils.data.DataLoader(mnist_val, batch_size, shuffle=False, num_workers=2),
            torch.utils.data.DataLoader(mnist_test, batch_size, shuffle=False, num_workers=2))
```

```python
# Function to evaluate the model's accuracy and loss on a given dataset
def evaluate_accuracy(net, data_iter, loss, device):
    """Compute the accuracy for a model on a dataset."""
    net.eval()  # Set the model to evaluation mode

    total_loss = 0
    total_hits = 0
    total_samples = 0
    with torch.no_grad():
        for X, y in data_iter:
            X, y = X.to(device), y.to(device)
            y_hat = net(X)
            l = loss(y_hat, y)
            total_loss += float(l)
            total_hits += sum(net(X).argmax(axis=1).type(y.dtype) == y)
            total_samples += y.numel()
    return float(total_loss) / len(data_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to train the model for one epoch
def train_epoch(net, train_iter, loss, optimizer, device):
    # Set the model to training mode
    net.train()
    # Sum of training loss, sum of training correct predictions, no. of examples
    total_loss = 0
    total_hits = 0
    total_samples = 0
    for X, y in train_iter:
        # Compute gradients and update parameters
        X, y = X.to(device), y.to(device)
        y_hat = net(X)
        l = loss(y_hat, y)
        # Using PyTorch built-in optimizer & loss criterion
        optimizer.zero_grad()
        l.backward()
        optimizer.step()
        total_loss += float(l)
        total_hits += sum(y_hat.argmax(axis=1).type(y.dtype) == y)
        total_samples += y.numel()
    # Return training loss and training accuracy
    return float(total_loss) / len(train_iter), float(total_hits) / total_samples  * 100
```

```python
# Function to select the device (GPU if available, otherwise CPU)
def try_gpu(i=0):
    """Return gpu(i) if exists, otherwise return cpu()."""
    if torch.cuda.device_count() >= i + 1:
        return torch.device(f'cuda:{i}')
    return torch.device('cpu')
```

```python
# Function to train the model over multiple epochs
def train(net, train_iter, val_iter, test_iter, num_epochs, lr, device):
    """Train a model."""
    train_loss_all = []
    train_acc_all = []
    val_loss_all = []
    val_acc_all = []
    def init_weights(m):
        if type(m) == nn.Linear or type(m) == nn.Conv2d:
            nn.init.xavier_uniform_(m.weight)
    net.apply(init_weights)
    print('Training on', device)
    net.to(device)
    optimizer = torch.optim.SGD(net.parameters(), lr=lr)
    loss = nn.CrossEntropyLoss()
    for epoch in range(num_epochs):
        train_loss, train_acc = train_epoch(net, train_iter, loss, optimizer, device)
        train_loss_all.append(train_loss)
        train_acc_all.append(train_acc)
        val_loss, val_acc = evaluate_accuracy(net, val_iter, loss, device)
        val_loss_all.append(val_loss)
        val_acc_all.append(val_acc)
        print(f'Epoch {epoch + 1}, Train loss {train_loss:.2f}, Train accuracy {train_acc:.2f}, Validation loss {val_loss:.2f}, Validation accuracy {val_acc:.2f}')
    test_loss, test_acc = evaluate_accuracy(net, test_iter, loss, device)
    print(f'Test loss {test_loss:.2f}, Test accuracy {test_acc:.2f}')

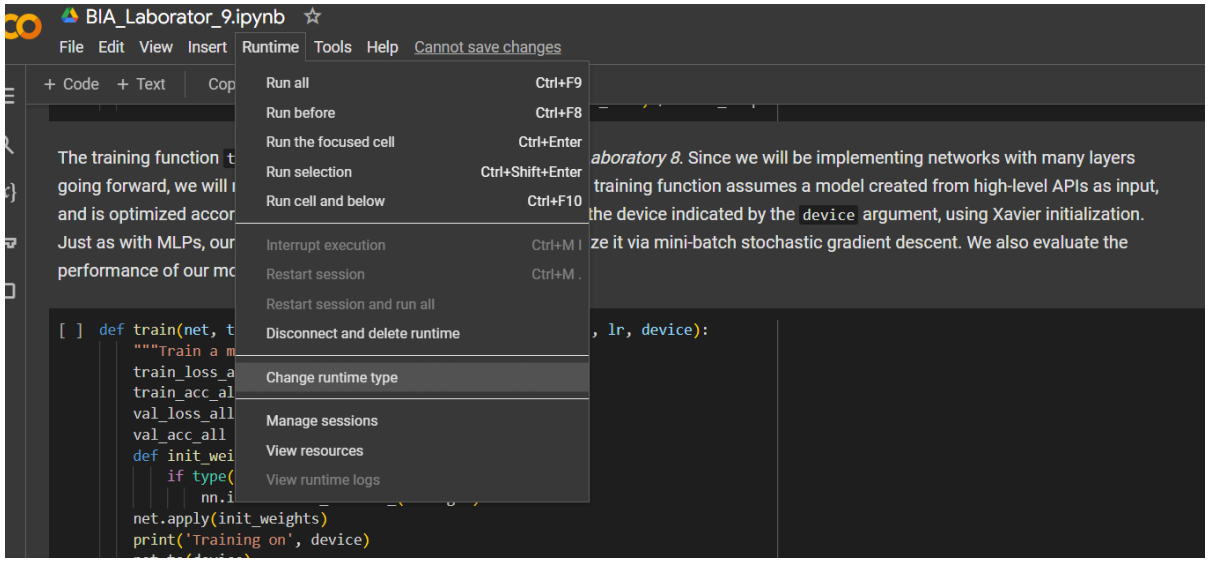
    return train_loss_all, train_acc_all, val_loss_all, val_acc_all
```

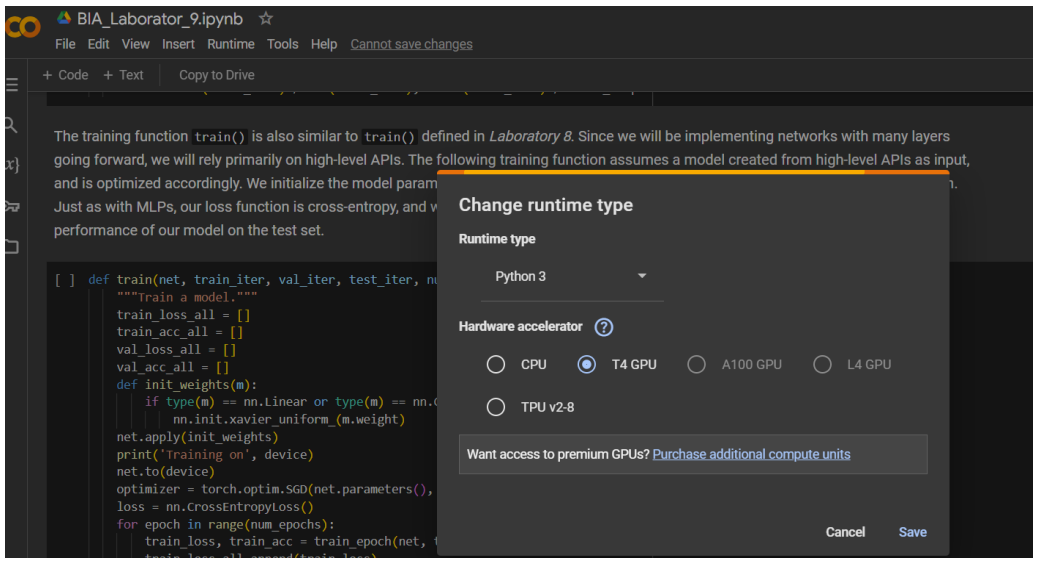
```python
def plot_loss(train_loss_all, val_loss_all):
    epochs = range(1, len(train_loss_all) + 1)
    plt.plot(epochs, train_loss_all, 'bo', label='Training loss')
    plt.plot(epochs, val_loss_all, 'b', label='Validation loss')
    plt.title('Training and validation loss')
    plt.xlabel('Epochs')
    plt.ylabel('Loss')
    plt.legend()
    plt.show()
```

```python
def plot_accuracy(train_acc_all, val_acc_all):
    epochs = range(1, len(train_acc_all) + 1)
    plt.plot(epochs, train_acc_all, 'bo', label='Training acc')
    plt.plot(epochs, val_acc_all, 'b', label='Validation acc')
    plt.title('Training and validation accuracy')
    plt.xlabel('Epochs')
    plt.ylabel('Accuracy')
    plt.legend()
    plt.show()
```

NOTE: don't forget to change the runtime from CPU to GPU:

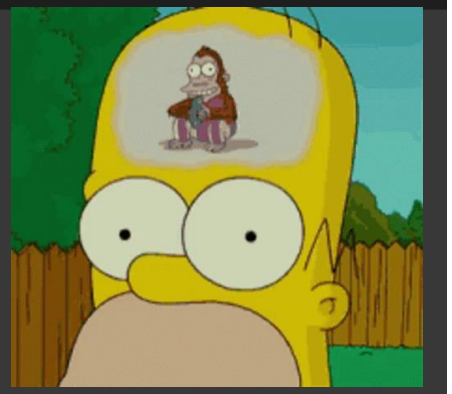




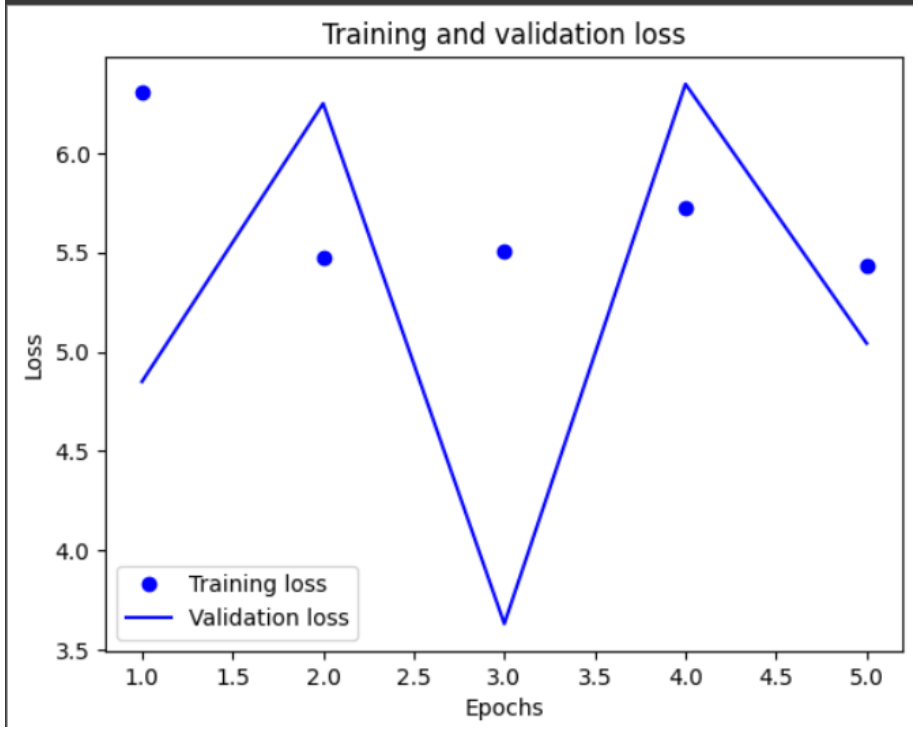


```
# Set hyperparameters for training
batch_size, lr, num_epochs = 256, 0.05, 5
train_iter, val_iter, test_iter = load_data_SVHN(batch_size) # Load training, validation, and test datasets
# Train the model and return loss/accuracy history
train_loss_all, train_acc_all, val_loss_all, val_acc_all = train(net, train_iter, val_iter, test_iter, num_epochs, lr, try_gpu())
```

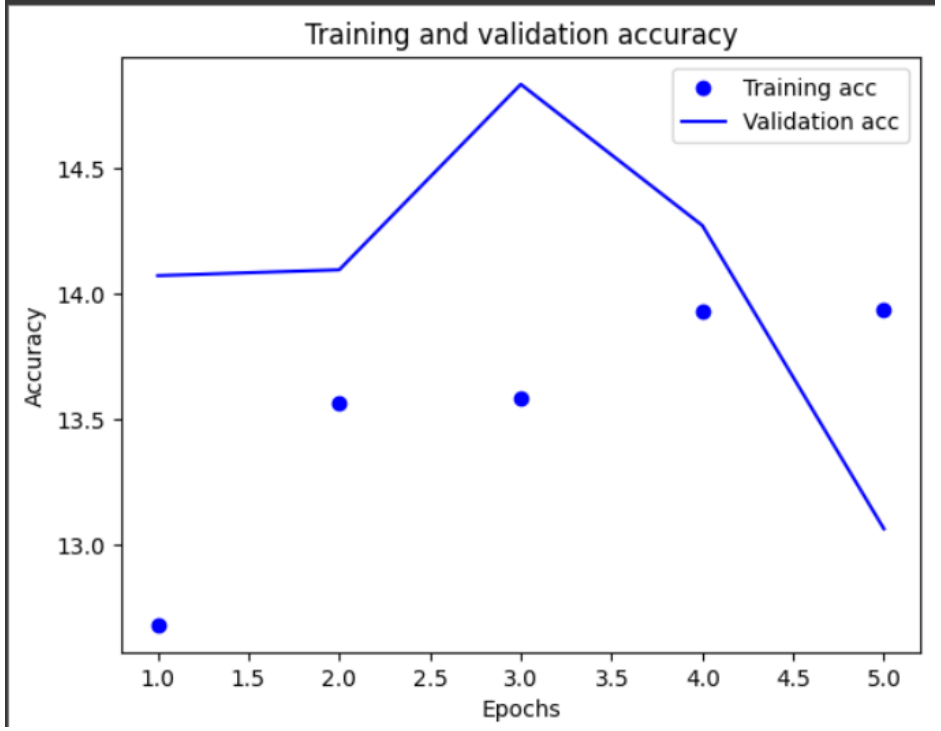
```
Downloading http://ufldl.stanford.edu/housenumbers/train_32x32.mat to ../data/train_32x32.mat
100%|██████████| 182M/182M [00:08<00:00, 22.6MB/s]
Downloading http://ufldl.stanford.edu/housenumbers/test_32x32.mat to ../data/test_32x32.mat
100%|██████████| 64.3M/64.3M [00:03<00:00, 20.4MB/s]
Training on cuda:0
Epoch 1, Train loss 6.31, Train accuracy 12.68, Validation loss 4.85, Validation accuracy 14.07
Epoch 2, Train loss 5.47, Train accuracy 13.56, Validation loss 6.25, Validation accuracy 14.10
Epoch 3, Train loss 5.50, Train accuracy 13.58, Validation loss 3.63, Validation accuracy 14.84
Epoch 4, Train loss 5.72, Train accuracy 13.93, Validation loss 6.35, Validation accuracy 14.27
Epoch 5, Train loss 5.44, Train accuracy 13.94, Validation loss 5.04, Validation accuracy 13.06
Test loss 5.15, Test accuracy 11.28
```



```
plot_loss(train_loss_all, val_loss_all)
```




```
plot_accuracy(train_acc_all, val_acc_all)
```



NOTE: the model is too dumb to learn this task, this is why we have so bad performance ☺

Optional

adaptation from *get_fashion_mnist_labels()* function from lab 8

```python
def get_svhn_labels(labels):
    """Return text labels for the SVHN dataset."""
    text_labels = ['0', '1', '2', '3', '4', '5', '6', '7', '8', '9']
    return [text_labels[int(i)] for i in labels]
```

```python
def show_images(imgs, num_rows, num_cols, titles=None, scale=1.5):
    """Plot a list of images."""
    figsize = (num_cols * scale, num_rows * scale)
    _, axes = plt.subplots(num_rows, num_cols, figsize=figsize)
    plt.tight_layout()
    axes = axes.flatten()
    for i, (ax, img) in enumerate(zip(axes, imgs)):
        if torch.is_tensor(img):
            # Tensor Image
            ax.imshow(img.numpy())
        else:
            # PIL Image
            ax.imshow(img)
        ax.axes.get_xaxis().set_visible(False)
        ax.axes.get_yaxis().set_visible(False)
        if titles:
            ax.set_title(titles[i])
    return axes
```

function from lab 8

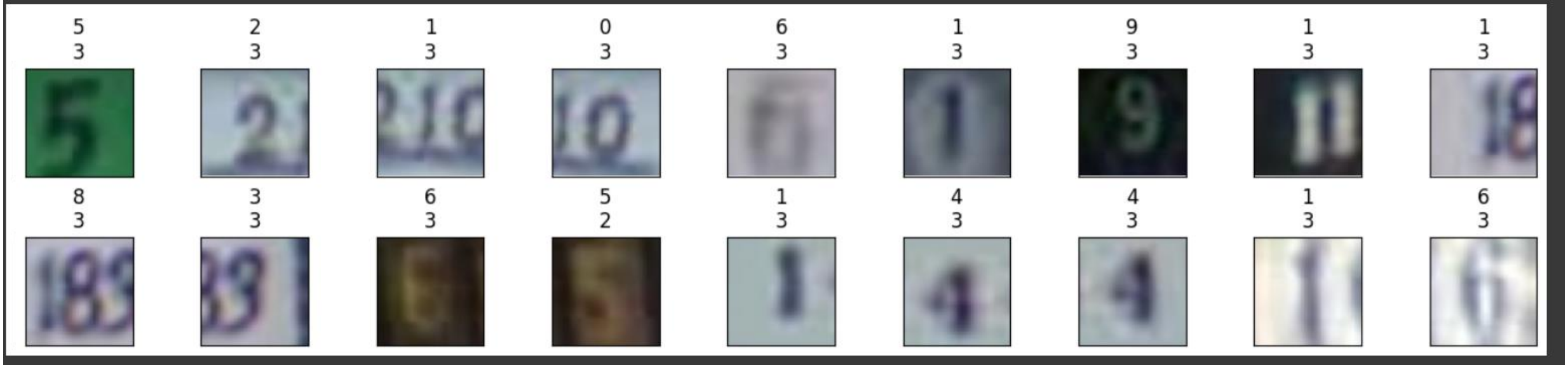
Optional

- use our model to classify some images. Given a series of images, we will compare their **actual labels (first line of text output)** and the **predictions from the model (second line of text output)**:

```
def predict(net, test_iter, n=18):
    """Predict labels."""
    for X, y in test_iter:
        break
    trues = get_svhn_labels(y)
    preds = get_svhn_labels(net(X.cuda()).argmax(axis=1))
    titles = [true +'\n' + pred for true, pred in zip(trues, preds)]
    show_images(X[0:n].permute(0, 2, 3, 1), 2, int(n/2), titles=titles[0:n])

predict(net, test_iter)
```

I needed to change these 2 things



Thank you !